A multi-agent automated research pipeline for wind engineering: Application to neural surrogates for urban wind field prediction

Liang Hu[a,*], Ahsan Kareem[a], Yanlin Guo[b], Jichuan Tang[c], Patrick Brewick[c]

[a]*Nathaz Modeling Lab, University of Notre Dame, Notre Dame, 46556, IN, USA*
[b]*Department of Civil and Environmental Engineering, Colorado State University, Fort Collins, 80523, CO, USA*
[c]*Department of Civil and Environmental Engineering and Earth Sciences, University of Notre Dame, Notre Dame, 46556, IN, USA*

Abstract

This study proposes an agentic wind engineering research system with an automated pipeline and demonstrates its use by developing neural-network surrogate model candidates for predicting pedestrian-level urban wind fields from building-landscape contour images. The system consists of multiple agents, including a planner, code generator, reviewer, controller, monitor, executor, and runner. These agents are driven by commercial LLMs, where each model may support multiple functions and each function may involve multiple models. The pipeline forms an iterative workflow that orchestrates these agents and mimics the human research process, while requiring only minimal human intervention after the research objective and constraints are specified.

The purpose of the pipeline is to model a system with specified inputs and outputs, such as the surrogate model for predicting urban wind fields in the example presented below. Initiated by a human researcher's idea, the pipeline begins with automatic model proposal generation by the planner, which synthesizes candidate models from accumulated knowledge, prior evidence, and external sources. The proposed candidates are then implemented by the code generator. After post-generation code review and smoke testing, feasible candidates are submitted by the executor to a remote HPC cluster for full training under a locked data and evaluation contract. The monitor tracks the submitted jobs, and the results are retrieved to the local machine and reviewed by the reviewer, producing structured evidence for the next iteration. This loop is gated by the controller, which enforces deterministic status transitions, retry and repair budgets, and artifact integrity, and is orchestrated by the runner state machine.

The pipeline is illustrated through a demonstrative example that seeks the best surrogate model for predicting urban wind fields from building-landscape contour images. Lu et al. (2025) is selected as the reproducible initial reference study (baseline), while Snaiki et al. (2026) is used as

*Corresponding author.
Email address: [user] (Liang Hu)

a virtual contrast representing the contribution of a human scientist (target). The automated pipeline is examined in three application modes: a grid mode with approximately 200 candidates generated at the start of the campaign and organized as 20 rounds of 10 candidates, a sequential mode with 30 iterative rounds of 12 candidates, and a hybrid mode with 24 iterative rounds (R0–R23) of 12 candidates initialized from the outcome of the grid mode. Across these modes, approximately 800 candidate configurations are trained and evaluated (about 200 grid-mode candidates, 319 sequential-mode and 280 hybrid-mode full-training records). The results show that the hybrid mode produces the largest number of high-performing candidates and identifies the best candidates outperforming the baseline model. The strongest verified hybrid-mode post hoc candidate is the anisotropic-kernel U-Net, which achieves a median R^2 of 0.7946 over 102 test cases across three random seeds, with approximately 6.88 million parameters. This corresponds to a 9.2% improvement over the baseline value of 0.728. However, the result remains below the virtual contrast target of $R^2 = 0.945$ reported by Snaiki et al. (2026). These findings indicate that, although the automated pipeline does not yet match the reported performance of human-designed two-stage refinement systems, it can provide a stronger starting point for subsequent human-led research.

Given the rapid development of LLMs, this pipeline is recommended as a useful prerequisite stage for research workflows that involve similar proposing-coding-evaluating loops in wind engineering and related fields.

Keywords: Agentic research, Multi-agent systems, Wind engineering, Urban wind fields, Neural surrogate models, Automated architecture search

---

1. Introduction

1.1. From AI for science to agentic research

Artificial intelligence has evolved from an auxiliary tool for prediction, optimization, and surrogate construction into an active participant that can plan experiments, write code, interpret results, and revise research strategies. This transition from AI for science to agentic research represents a qualitative shift in the unit of automation (Wei et al., 2025; Xin et al., 2025). Earlier workflows automated isolated computational tasks, such as fitting a surrogate model or tuning a hyperparameter grid, while the surrounding research loop of proposing ideas, implementing experiments, interpreting evidence, and deciding next steps remained human-operated. Agentic systems attempt to automate larger portions of this loop, translating a research goal into candidate experiments, creating executable artifacts, checking their validity, running evaluations, collecting evidence, and using the accumulated knowledge to choose the next action.

The distinction between conventional AI-assisted research and agentic research can be understood through the feedback structure. In conventional use, an AI model produces a single

prediction or generation, and the human researcher decides whether and how to iterate. In agentic research, the system itself manages a multi-step loop: it generates a hypothesis or candidate, implements it as executable code, submits it for evaluation, collects and interprets the result, and uses that interpretation to generate the next candidate. The human researcher defines the objective, constraints, and evaluation contract at the outset, but the iterative cycle proceeds with minimal intervention. This shift from single-step assistance to closed-loop research automation is what distinguishes agentic systems from earlier AI-for-science tools and creates both their promise and their risks.

This larger scope is attractive for scientific and engineering research because many projects contain repeated propose-code-evaluate cycles, but it also creates new risks. The system must distinguish genuine scientific failure from implementation failure, prevent invalid outputs from contaminating the experimental record, manage expensive computation responsibly, and make evidence-based decisions without hiding uncertainty behind fluent language.

1.2. Autonomous research systems: landscape and maturity

The past three years have seen rapid development of autonomous and semi-autonomous research systems, which can be organized by maturity tier (Tie et al., 2025). At the most mature level, established single-step tools augment individual research activities without closing the full research loop: literature search platforms (Semantic Scholar, Elicit), writing assistants (Writefull, Paperpal), systematic review screening tools (ASReview, Rayyan), automated machine learning frameworks (Auto-sklearn, AutoGluon, NNI), and code generation assistants (GitHub Copilot, Claude, Cursor). These tools are widely adopted but each addresses only one step of the research process.

At an intermediate level, several systems have demonstrated narrow-domain research closures in which the AI manages a multi-step cycle from hypothesis through evaluation. The AI Scientist (Lu, C. et al., 2024; Yamada et al., 2025; Lu, C. et al., 2026) has progressed from template-based machine learning research to a system described in a human-authored *Nature* article in 2026, after achieving workshop-level paper acceptance at ICLR 2025. The Google AI Co-Scientist (Gottweis et al., 2025) uses a multi-agent coalition and has produced verified results in drug repurposing. Coscientist (Boiko et al., 2023) was an early high-profile LLM-controlled physical chemistry laboratory automation system, using GPT-4 to drive liquid handlers for automated cross-coupling reactions. ChemCrow (Bran et al., 2024) integrated 18 expert-designed chemistry tools for organic synthesis and drug discovery. More recently, Camyla (Gao et al., 2026) achieved autonomous medical image segmentation research across 31 datasets, exceeding the nnU-Net baseline on 24 of them. AutoSOTA (Li et al., 2026) used an 8-agent system to produce 105 new state-of-the-art models.

A closely related line of work couples LLM generation with automated evaluators in an iterative loop. FunSearch (Romera-Paredes et al., 2024) paired a pretrained LLM with a systematic

evaluator, scoring generated programs and feeding high-scoring programs back into subsequent generation, and AlphaEvolve (Novikov et al., 2025) extended this generate–evaluate–iterate pattern to an LLM coding agent that modifies code under evaluator feedback. For neural architecture design, EvoPrompting (Chen et al., 2023) used a language model to generate code-level architecture candidates that were trained, evaluated, and selectively fed back to the generator. In machine-learning engineering, AIDE (Jiang et al., 2025) organizes candidate scripts as a search tree that is drafted, debugged, and improved according to evaluator scores, and benchmarks such as MLAgentBench (Huang et al., 2024) and MLE-bench (Chan et al., 2025) evaluate language agents on machine-learning experimentation and Kaggle-style engineering tasks. These systems share the generate–evaluate–revise mechanism used by the sequential and hybrid modes in this study, but they were demonstrated on mathematical, algorithmic, or general machine-learning tasks rather than on engineering surrogate development.

A growing family of research-assistant systems, including Agent Laboratory (Schmidgall et al., 2025), DeepScientist (Weng et al., 2025), and OpenResearcher (Zheng et al., 2024), organizes AI components into staged workflows for literature review, experimentation, retrieval, or report generation. These designs reduce the risk that a single model's biases dominate the research output and improve auditability. However, most of these systems have been demonstrated only on machine learning or software benchmarks with fast feedback loops and low per-experiment cost. Their applicability to engineering or physical-science problems with longer evaluation cycles and higher failure rates remains largely untested.

The *Nature Machine Intelligence* Comment "Towards agentic science for advancing scientific discovery" (Xin et al., 2025) framed the opportunity while warning against premature claims of human-level research capability. The comprehensive survey by Xu et al. (Wei et al., 2025) mapped the transition from AI for science to agentic science, categorizing systems by autonomy level and identifying open challenges in reliability, reproducibility, and trust. Current evidence is still closer to AI-assisted and AI-orchestrated research than to fully independent scientific authorship; the most apt characterization is that these systems function as diligent junior research engineers rather than independent scientists.

1.3. Concerns and limitations of agentic research

Agentic research systems face several categories of concerns that must be acknowledged transparently. These can be grouped into three clusters.

The first cluster concerns scientific validity. Novelty assessment failure remains pervasive: systems can mark known techniques as original contributions, and self-generated novelty checks are unreliable because the same model that proposes an idea is poorly positioned to assess whether it already exists. LLMs also produce fabricated or inaccurate references: in one controlled GPT-4o study, 19.9% of generated citations were fabricated and 45.4% of the real citations contained bibliographic errors (Linardon et al., 2025). The Meta Galactica system, introduced as a scientific

language model (Taylor et al., 2022), had its public demo rapidly withdrawn after users found that it produced authoritative-sounding but false or biased scientific content (Heaven, 2022). Scholarly publishing is already under stress from paper mills and peer-review fraud, with more than 10,000 research papers retracted in 2023 (Van Noorden, 2023). A scientific monoculture risk also emerges if researchers use identical models, search tools, and ranking heuristics, potentially narrowing the diversity of inquiry and communication style.

The second cluster concerns scholarly integrity. Author attribution and accountability remain unresolved when AI generates hypotheses, code, and text. Major publishers now require human accountability for AI-assisted manuscripts and do not treat AI tools as responsible authors (Nature Portfolio, 2026), but the practical boundaries between human authorship and AI contribution are not well defined. Dual-use concerns are acute for systems that interact with physical instruments or chemical reagents; the Coscientist publication withheld full code and prompts due to security considerations (Boiko et al., 2023). For peer review, an external analysis of ICLR 2026 reviews estimated that approximately 21% were fully AI-generated, a finding later reported by *Nature* (Pangram Labs, 2025; Nature, 2025). This signals a feedback loop that threatens review integrity.

The third cluster concerns operational accountability. Automation bias is a concern even when the system produces correct outputs: researchers may over-trust AI-generated results without independent verification, and data lineage through automated pipelines may be difficult to trace without explicit artifact management.

The above concerns do not invalidate the potential of agentic research, but they motivate careful system design with deterministic gates, auditable artifact trails, and separation of model-generated evidence from human interpretation. The pipeline described in this study is designed with these safeguards as primary requirements.

1.4. Applications in wind engineering and related fields

Wind engineering is a natural but underdeveloped application area for agentic research systems and particularly automated research pipelines. However, the field already uses computational fluid dynamics (CFD), wind-tunnel testing, reduced-order modeling, uncertainty quantification, structural response prediction, and data-driven surrogates extensively, and each of these activities contains repeated propose-configure-evaluate cycles that could benefit from systematic automation.

In the finite-element analysis domain, several recent systems have explored LLM-driven automation, including FeaGPT (Qi et al., 2025) and MechAgents (Ni and Buehler, 2024) for LLM-assisted FEA setup, execution, and result interpretation. In the CFD domain, a growing number of agent-based systems have emerged: Foam-Agent 2.0 (Yue et al., 2025) demonstrated multi-agent CFD automation and was presented at NeurIPS 2025; MetaOpenFOAM 2.0 (Chen et al., 2025) and AutoCFD (Dong et al., 2025) provide multi-agent or fine-tuned-LLM OpenFOAM automation; OpenFOAMGPT (Pandey et al., 2025) combines retrieval of OpenFOAM tutorial information with iterative error correction; and Feng et al. (Feng et al., 2025) proposed an LLM-powered discovery

framework for experimental fluid mechanics. In scientific machine learning, frameworks such as NVIDIA PhysicsNeMo (NVIDIA, 2026), DeepXDE (Lu et al., 2021), and benchmark suites including FlowBench (Tali et al., 2024) and PDEBench (Takamoto et al., 2022) provide standardized environments for surrogate development, but they do not close the research loop from hypothesis through evaluation and revision.

However, all existing engineering agent systems related to wind engineering generally remain limited to configuration assistance or execution automation for individual tasks, and no integrated autonomous pipeline has yet been documented that closes the full idea-to-simulation-to-analysis research loop. This limitation is particularly consequential in wind engineering because simulation and surrogate-development workflows differ fundamentally from the short-cycle benchmarks common in computer science and machine learning. Candidate evaluation in wind engineering often requires day-to-week feedback loops, involves more failure modes, incurs higher computational cost, and is harder to validate automatically. As a result, each candidate is expensive to test, making systematic autonomous search not merely convenient but strategically valuable for accelerating wind engineering research.

1.5. Urban wind field simulation and surrogates

Pedestrian-level wind conditions depend strongly on urban morphology, building height variation, street-canyon structure, and local flow acceleration around obstacles. Accurate prediction of these conditions is essential for urban planning, outdoor comfort assessment, pollutant dispersion modeling, and wind-energy siting. High-fidelity large-eddy simulation (LES) or comparable CFD methods can represent these effects with physical fidelity, but they are too expensive for routine design-space exploration at city scale. A single LES case on a domain covering several city blocks may require hours to days of computation on modern GPU or CPU clusters, making parametric studies over hundreds of urban configurations impractical with direct simulation alone.

Neural surrogates address this bottleneck by learning a mapping from building-landscape fields directly to wind-speed fields, providing rapid predictions that approximate the CFD output at a fraction of the computational cost. The architecture space is large: depth, channel count, skip-connection design, attention mechanisms, spectral or Fourier layers, normalization strategy, and data augmentation all interact in ways that are difficult to predict from first principles.

The UrbanTALES dataset (Nazarian et al., 2025) provides a standardized LES resource for urban canopy wind studies. The machine-learning subset used in the present demonstrative problem follows Lu et al. (2025) and contains 512 LES cases (224 idealized and 288 realistic urban configurations) with pedestrian-level 2D time-averaged wind speed at $z = 1.5$ m height. Lu et al. demonstrated that a U-Net architecture trained on this subset can achieve reasonable prediction accuracy, establishing a reproducible baseline. Snaiki et al. (2026) subsequently proposed a hierarchical predictor-refiner approach that uses the Lu-style U-Net as a first stage and a conditional generative adversarial network as a second-stage refinement, reporting substantially higher

accuracy on the same dataset.

The urban wind field surrogate problem is well suited to an automated research pipeline for several reasons. The input and output are structured arrays on a regular spatial grid, making them compatible with standard convolutional and operator architectures. The performance metrics (R^2, RMSE, MAE computed per case and aggregated) are computable without manual interpretation. Baseline implementations exist in the published literature and have been reproduced. Training is expensive enough (4 to 8 hours per candidate) to make search strategy meaningful, and the architecture design space is sufficiently large that systematic exploration can discover configurations that a human researcher would be unlikely to try manually.

1.6. Scope and organization of this study

This study develops and documents a multi-agent automated research system for neural surrogate development in wind engineering. The scientific target is not to prove that an automated agent has replaced a human wind engineer. The target is more practical: given a reproducible starting study, a controlled data and metric contract, and a budget for many candidate trainings, the pipeline should produce a stronger set of starting architectures, reports, and hypotheses than a human researcher would obtain from a single baseline run. Lu et al. (2025) is used as the reproducible initial reference because it provides the starting urban wind surrogate workflow and has been successfully reproduced. Snaiki et al. (2026), which proposes a hierarchical predictor-refiner method on the same problem, is used only as a contextual human-designed contrast because its reported evaluation does not establish a same-split, same-seed, same-code comparison.

The remainder of this study is organized as follows. Section 2 describes the seven functional agents and their multi-model implementation. Section 3 describes the closed-loop automated pipeline, including proposal generation, code generation, post-generation review, smoke testing, the test-retry-repair loop, full training, result review, and round transition. Section 4 introduces the demonstrative problem, presents the three application modes (grid, sequential, and hybrid), and reports the results. Section 5 discusses limitations, deficiencies, and broader implications. Section 6 presents concluding remarks. All materials, codes, and AI-generated reports produced during this research are intended to be shared online after publication, subject to appropriate redaction of private infrastructure details.

2. Multi-agent research system

2.1. Role-based system design

In this study, the automated research system aims to model a system with specified inputs and outputs, such as the model for predicting urban wind fields in the example presented below. This objective is pursued through a propose-code-evaluate loop, in which model candidates are first proposed, then realized, for example through code implementation, and finally evaluated against

predefined target constraints. Through iterative looping, the system is expected to progressively reduce the modeling error. This error must be observable, quantifiable, and comparable across candidates so that model improvement can be assessed objectively.

The automated research system is organized around seven functional agents: a planner, code generator, reviewer, controller, monitor, executor, and runner. These agents are defined by their responsibilities rather than by a one-to-one mapping to specific LLMs. In practice, multiple commercial LLMs may be combined within a single agent role, and one model may serve multiple roles when its strengths match the required tasks. This separation between role and model is a deliberate design choice. It allows the workflow to be described in terms of public functions, inputs, outputs, and gates, while treating individual model providers as interchangeable implementation choices that can be updated independently. It also enables multi-model competition, in which several LLMs propose candidates in parallel and a synthesis step integrates their proposals by exploiting the strengths of different models while reducing dependence on the biases or weaknesses of any single model. Finally, it allows the system to assign models according to task-specific strengths, such as using a code-oriented model for the code generator role and a reasoning-oriented model for the reviewer role.

Figure 1 illustrates the relationships among the seven agents and their connections to the knowledge base and computational backend.

2.2. Runner

The runner is the central workflow orchestrator of the state machine. It maintains a mapping from phase names to handler functions and a transition table that defines the permissible phase transitions. For a single round, the valid phase sequence is: proposal, code generation, post-code-generation review, smoke submission, monitoring, collection, smoke classification, AI fix, repair code generation, full submission, monitoring, collection, review, and next round. Not all phases are visited in every round. For example, if all candidates pass smoke testing, the smoke-classification and AI-fix phases are skipped. These phases are detailed in Section 3.

The runner provides deterministic workflow governance that confines LLM-driven agents to their designated functions. Based on a scheduled task, it examines the status gates enforced by the controller and determines when the workflow should move to the next phase.

2.3. Planner

The planner converts a human research objective into a batch of model candidate experiments. It receives the task definition, the locked data and evaluation contract, the admissible configuration space, hard constraints on trainability and resources, the current campaign state, the recent knowledge bundle, the model-family summary, the failure taxonomy, the hypothesis registry, and selected review artifacts from prior rounds. This context package ensures that every new proposal

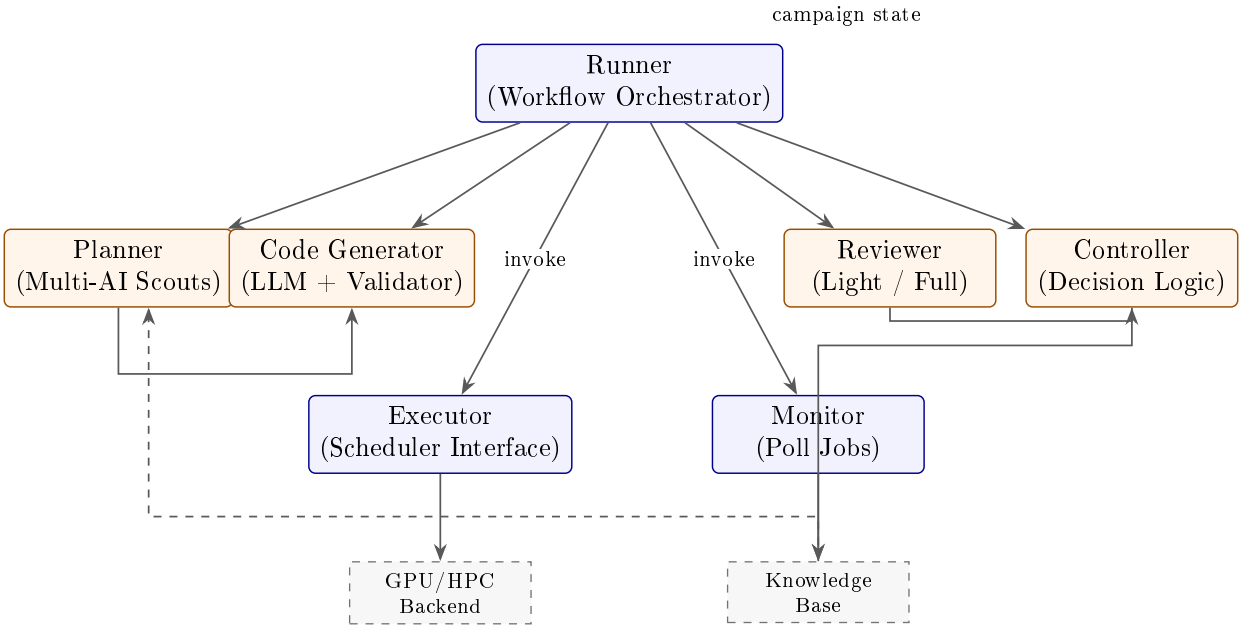


Figure 1: Multi-agent roles in the automated research system. Orange boxes denote LLM-driven roles; blue boxes denote deterministic infrastructure. The dashed arrow indicates accumulated knowledge feeding back to the planner for subsequent rounds.

is traceable to either accumulated campaign evidence or an explicitly identified exploration hypothesis. The planner then proceeds with four stages: external discovery, multi-model scouting, synthesis, and deterministic gating.

Before ordinary scouting begins, an external web-scout stage supplies curated mechanism material from outside the campaign history. A Google Gemini source scout searches adjacent fields (in the demonstrative example: CFD field prediction, neural operators, dense spatial regression, topology-to-field learning, super-resolution, and boundary refinement). An OpenAI Codex web-idea scout converts retained sources into implementation-light mechanism ideas, and an Anthropic Claude local-review step filters these ideas against the context of the campaign. The stage produces a quality report recording valid and weak sources, topic-cluster coverage, missing clusters for the next round, retry hints, and query-yield statistics. A quality gate determines whether the web-scout output meets a minimum coverage threshold; rounds that fall short proceed without web context rather than injecting low-quality material. Web-scouts are not permitted to produce final training configurations. Their output prevents the planner from repeatedly sampling only mechanisms already present in the local campaign history.

The ordinary scout stage then invokes multiple LLM scouts concurrently. Primary scouts (Claude, Codex, Gemini) generate the main candidate pool, while diversity scouts (DeepSeek, MiMo, Grok) contribute lower-weight auxiliary ideas or critiques. Each scout receives the candidate model library, search space, hard constraints, recent evidence, failure patterns, hypothesis records, and reviewed web-scout ideas when available. Primary scouts return structured candidate configurations with rationale fields. Diversity scouts are treated as mechanism sources, objections, or repair suggestions rather than as equal-weight configuration owners. Reviewed web ideas enter the scout prompt as optional context, not directives; scouts may adopt, adapt, or discard them based on campaign evidence.

A synthesis model (Claude) then merges primary proposals with selected web-derived and diversity-scout mechanisms to form the final batch, typically 12 experiments per round. The synthesis prompt explicitly distinguishes exploitation from exploration: exploitation proposals refine or replicate promising model families under controlled perturbations, whereas exploration proposals must introduce a new model value, a new composition, or a lightweight mechanism transfer compatible with existing contracts. When web-scout ideas pass the quality gate, they serve as preferred raw material for exploration slots, provided they can be converted into feasible candidates without new data, solvers, labels, or metrics.

The batch is passed through a deterministic quality gate (Codex) before submission. The gate checks schema validity, duplicate or near-duplicate signatures, seed and batch-size locks, approved loss options, input-feature contracts, parameter-count and resource guards, evidence-reference fields, and the balance between exploitation and exploration. Each accepted candidate records source confidence, synthesis influence, evidence references, comparator links, and belief-update rules so that later reviewers can determine why it was proposed and what outcome would

support or refute its hypothesis. A belief-update rule takes the form: "if metric M improves by at least δ relative to comparator C, increase confidence in hypothesis H; otherwise decrease it and flag the mechanism for review." These rules give the reviewer explicit criteria for updating the hypothesis registry after results return.

This four-stage separation (external discovery, multi-model scouting, synthesis, and deterministic gating) reflects a practical lesson from pipeline development: a planner that only performs local refinement collapses into repeated width, depth, or learning-rate variations, while one that overweights novelty produces fragile or resource-infeasible candidates. The hybrid design is not a formal Bayesian acquisition function but an auditable proposal system that balances campaign evidence, external mechanism discovery, resource feasibility, and controlled exploration.

2.4. Code generator

The code generator translates planner proposals into executable model artifacts. For each proposed model design, it produces a standalone module implementation that must satisfy an explicit interface contract. For example, in the following demonstrative example, the contract requires that the module accepts a tensor of shape (B, C, H, W) where B is the batch size, C is the number of input channels (determined by the selected input features), and $H = W = 640$ is the spatial resolution; produces an output tensor of shape $(B, 1, H, W)$; uses GroupNorm instead of BatchNorm for stability with small batch sizes; applies reflection padding instead of zero padding; and does not call unsafe numerical operations that would mask errors.

The code generation process operates in a generate-review-refine cycle of up to three iterations. First, an LLM generates a standalone module implementation from the candidate model's specification. Second, three automated validation checks are applied: syntax validation (import and parse), shape validation (a forward pass with a test tensor to verify input-output dimensions), and parameter count validation (within 20% of the target specified in the model catalog). Third, a review LLM examines the generated code for correctness and adherence to constraints. If issues are found, the code is sent back for another generation-validation cycle.

This role provides the bridge between open-ended model ideation by the planner and controlled scientific experimentation. The interface contract is essential because the pipeline intentionally allows the planner to explore model designs that may not exist in any published library, while the downstream validation ensures that only executable and contract-compatible candidates consume full training resources.

2.5. Reviewer

The reviewer serves a dual role in the pipeline and operates in two modes: a light mode as a debugging assistant and a full mode as a scientific critic.

The light mode is used to diagnose issues in model experiments. It is typically activated during the repair phase, when a failed model experiment is sent to the reviewer for diagnosis. Two LLM

agents analyze the failure: one examines the error logs and model details to identify the root cause, while the other proposes concrete fixes. Their diagnoses are then merged into a fix plan that specifies the identified problem, the repair strategy, and safe configuration patches.

The full mode is activated after a round of model experiments has concluded, for example, when all full-training runs in a round are complete. In this mode, the reviewer analyzes the collective results using two LLMs. An analytical model reviews performance metrics and trends, while a synthesis model consolidates the findings into strategic recommendations. The review output includes a ranking of all experiments by the primary metric, identification of promising candidate model families, detection of stagnation where applicable, and recommendations for the next round.

The reviewer's round-level outputs serve as the primary evidence input for the planner's next-round proposals, creating a chain of accountability. The reviewer must justify its recommendations by citing specific results, and the planner must link its proposals to reviewer findings. When preparing its analysis, the reviewer maintains a recent-round look-back window of five rounds. Earlier rounds are represented only through the aggregated knowledge bundle, which preserves family-level statistics but omits individual experiment details. This is a pragmatic compromise between context quality and prompt-length constraints, and its limitations are discussed in Section 5.5.

2.6. Controller

The controller is the primary deterministic gate in the workflow. It is invoked after result collection to determine the next action for each experiment and for the round as a whole. Its operation involves no LLM calls; it is fully deterministic.

For each experiment, the controller collapses retry and repair attempts to the latest result per base experiment identifier, invokes a failure classifier to obtain a classification and recommended action, and checks attempt limits using the attempt manifest. The per-experiment action space includes five categories: PASS (smoke test completed successfully; eligible for full training), REPAIR (a code error was detected; sent to diagnosis and AI-generated repair), RETRY (a transient failure occurred; resubmitted), WAIT (still running), and AUTO_FAIL (attempt budgets exhausted; marked as terminal).

The failure classifier applies rules in priority order: completed status with valid metrics yields PASS; still running or queued yields WAIT; detected code error patterns yield CODE_ERROR with action REPAIR; detected transient error patterns yield TRANSIENT_ERROR with action RETRY; resource infeasibility yields AUTO_FAIL; and otherwise the failure is classified as unknown with action RETRY up to the attempt limit.

At the round level, the controller determines the phase transition based on the distribution of per-experiment actions. This deterministic gating prevents incomplete or failed candidates from being treated as successful experiments, limits the risk that free-form LLM outputs directly control expensive computation, and ensures that the campaign record remains auditable.

2.7. Executor

The executor is the interface between the research workflow and the HPC backend. It converts validated candidate configurations into scheduler-ready jobs: preparing run directories, writing submission scripts (Slurm, PBS, HTCondor, or a comparable scheduler), staging model source files and configuration files to remote paths, submitting jobs through the scheduler interface, and recording the returned job identifiers and attempt identifiers in the campaign state. Each job is tagged with a unique experiment identifier and an attempt identifier that distinguishes retries from new candidates.

Resource-aware routing is central to the executor's value on shared HPC systems. The executor encodes GPU class, wall time, memory expectations, batch-size policy, retry limits, and output paths for each candidate. Candidates that exceed ordinary memory thresholds are escalated to high-memory GPU nodes, while standard candidates remain on the default pool. This distinction matters because a training failure on a shared cluster may reflect scheduler placement or memory class rather than a flaw in the candidate itself; without explicit routing, such failures would contaminate the reviewer's evidence and bias subsequent planner rounds. When a job fails due to a resource mismatch, for example, an out-of-memory error on a standard node, the executor can resubmit to a higher-memory class under a new attempt identifier without altering the candidate configuration, preserving a clean separation between infrastructure faults and scientific outcomes.

Before launching any job, the executor performs staging and submission checks. It verifies that required source files, configuration files, data paths, and output directories exist on the target system, avoids overwriting previous attempts, and writes a reproducible command record that captures the exact submission script, environment variables, and file checksums. In the implemented campaigns, the executor uses deterministic templates for the scheduler and environment, with LLM assistance limited to generating or adapting submission scripts when the backend changes. After submission, the executor does not interpret scientific results. It hands job identifiers and expected output locations to the monitor, which performs status polling and log inspection.

2.8. Monitor

Job submission is handled by the executor, whereas job-status polling is handled by the monitor. After a job is submitted, the monitor takes over and polls the status of submitted jobs when invoked by the runner. It queries the scheduled computational backend for job states, including queued, running, completed, held, and evicted, and checks for the presence of completion sentinel files. For running jobs, the monitor can also detect crash conditions by examining log files for error patterns. Early crash detection allows the workflow to proceed to failure classification without waiting for the scheduler timeout.

3. Automated pipeline

3.1. Overview of the closed-loop workflow

The pipeline converts a human research idea into an iterative propose-code-test-train-review cycle. Figure 2 illustrates the full pipeline structure. After the objective and constraints are defined, the system executes the following sequence in each round:

1. The planner proposes a batch of candidate model experiments (including model-design proposals, hyperparameter variants, and neutral or control models in the demonstrative example; typically 12 per round).
2. The code generator implements each proposed model design as executable code.
3. A post-generation reviewer checks the generated artifacts before any training.
4. Candidates are submitted for smoke testing (short training runs).
5. Failed candidates enter the test-retry-repair loop.
6. Surviving candidates are submitted for full training.
7. The monitor tracks job status; the collector retrieves structured results.
8. The reviewer produces a round summary with evidence-based recommendations.
9. The controller gates the transition to the next round.
10. The knowledge base is updated; the planner receives the new evidence for the next round.

This ordering is essential because the planner is intentionally allowed to explore model designs that may not already exist in any registry, while the downstream gates ensure that only executable and contract-compatible candidates consume full training resources.

3.2. State machine and campaign artifacts

The pipeline maintains persistent state across rounds through a structured set of campaign artifacts. The runner writes and reads a campaign state file that records the current phase, round number, and per-experiment status. Each experiment has a unique identifier, and the attempt manifest tracks how many retries and repairs have been used for each base experiment, preventing indefinite cycling. The knowledge base is rebuilt after each round from the updated campaign history, producing an all-results summary, a model-family summary with aggregated statistics, a failure taxonomy organized by model design, a hypothesis registry linking scout proposals to outcomes, and a compact knowledge bundle formatted for LLM context.

All generated artifacts, including proposed configurations, generated model code, smoke logs, training metrics, reviewer summaries, and controller decisions, are stored in structured directories. This artifact structure is central to reproducibility: a completed campaign can be audited from proposal through final evidence without relying on the LLMs that produced the intermediate outputs.

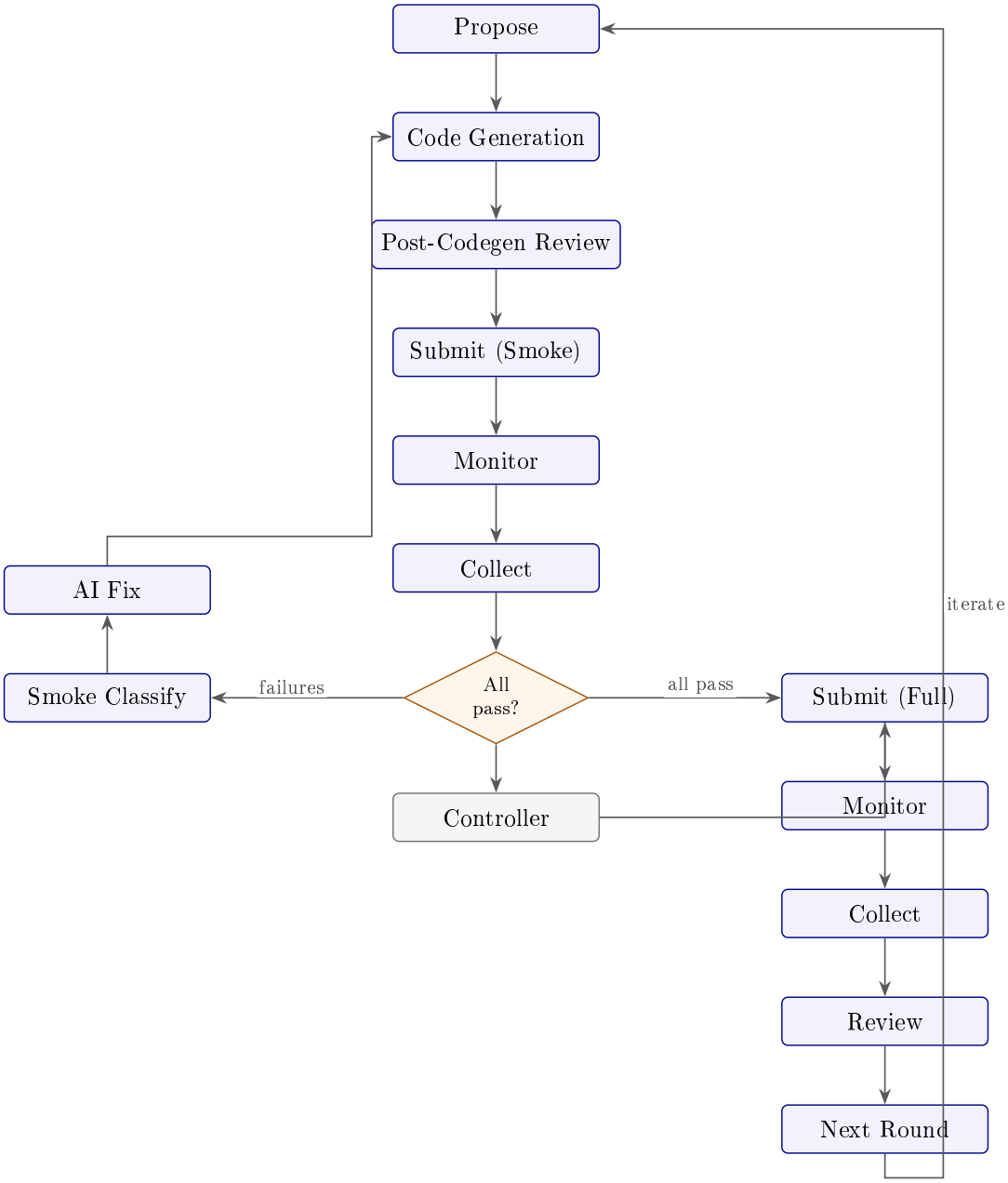


Figure 2: State-machine diagram of the automated propose-code-test-train-review pipeline. The runner advances this state machine by invoking the handler for the current phase, while the controller makes deterministic pass, retry, repair, or stop decisions after collected results are available. Each round follows the main path from proposal through full training and review; failed smoke tests enter the repair branch before returning to code generation. The loop from the next-round node back to proposal closes the iterative cycle.

3.3. Proposal and code-generation phase

The round begins with the planner generating a batch of experiment proposals, typically 12 candidates. The multi-model scouting mechanism described in Section 2.3 produces diverse proposals that are synthesized and validated against hard constraints. The planner context includes the candidate model library, the campaign history, the knowledge bundle, hard constraint rules, and the round review from the previous round.

For each proposed model candidate, the code generator produces executable code following the generate-review-refine cycle described in Section 2.4. If a proposed candidate model matches an existing entry in the shared model library, the code generator extracts the relevant class rather than generating from scratch.

3.4. Post-generation review

A post-generation review step uses an LLM agent to examine the generated code for subtle issues that automated validation might miss, such as incorrect activation functions, missing normalization layers, potential numerical instability, or violations of the modeling contract that do not manifest as immediate execution errors. This step is distinct from the smoke test: it is a code-level analysis, while the smoke test is an execution-level verification.

3.5. Smoke testing

Validated model code is submitted for smoke testing: a short training run of 20 epochs. The purpose of smoke testing is strictly to verify that the code runs to completion without errors on real training data. It is not a performance evaluation. Smoke tests detect runtime errors, tensor shape mismatches, memory exhaustion, numerical divergence, and other issues that static code analysis and single-tensor forward passes cannot catch.

Jobs are submitted to the scheduled computational backend by the executor, and the workflow enters the monitor phase. The monitor polls job status and checks log files for early crash indicators. When all smoke jobs reach a terminal state, the executor retrieves metrics and log excerpts from each completed run.

3.6. Test-retry-repair loop

The test-retry-repair loop is one of the technically difficult parts of the workflow because failures have different meanings and require different responses. This loop, illustrated in Figure 3, distinguishes among several failure categories and routes each to the appropriate response.

A syntax error, missing dependency, tensor-size mismatch, or invalid reshape is classified as a code defect and routed to diagnosis and repair. A transient backend interruption, GPU memory exhaustion, or scheduler eviction is classified as a transient error and handled by resubmission, potentially with resource escalation to a higher-memory GPU node. For experiments that failed

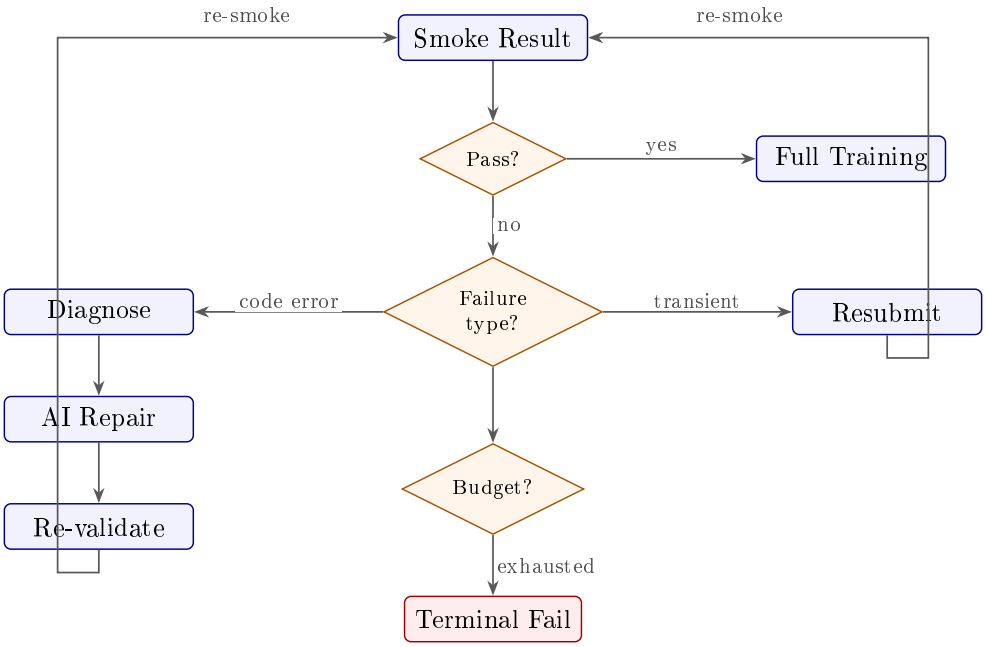


Figure 3: Test-retry-repair loop governed by the runner state machine and controller decisions. The monitor supplies job status and log evidence; code errors are diagnosed by the reviewer and repaired by the code generator before re-entering smoke testing. Transient failures, including memory or scheduling failures, are resubmitted directly. Both routes are subject to configurable attempt budgets; exhausted budgets result in terminal failure.

due to GPU memory constraints during smoke testing, the executor can escalate them to higher-memory GPU nodes if available. A configuration that violates campaign constraints, exceeds the attempt budget, or repeatedly fails the same gate is marked as terminal.

The attempt manifest records the number of retries and repairs used for each base experiment, with configurable limits: a maximum of 3 retries for transient failures, a maximum of 2 repairs for code errors, and a maximum of 5 total attempts per experiment. These budgets prevent a candidate from cycling indefinitely through LLM repair.

3.7. Full running

Once all experiments have either passed smoke testing or been marked as terminal failures, the surviving candidates are submitted for full running. In the current demonstration, running refers to full model training that uses 200 epochs under the same locked data split, feature contract, loss options, and evaluation scripts specified at the start of the campaign. The same submission, monitoring, and collection infrastructure is used as for smoke testing, but with longer expected runtimes (typically 4 to 8 hours per candidate on available GPU nodes).

3.8. Review and next-round

After all full-running results for a round are collected, the reviewer produces a round summary that ranks candidates, identifies failure modes, detects search stagnation where applicable, and recommends directions for the next round. The knowledge module then rebuilds its artifacts from the updated campaign history. The round counter is incremented, and the workflow returns to the proposal phase for the next round. In this way, knowledge from earlier rounds is preserved and reused, while each transition remains gated by explicit state and artifact checks.

4. Demonstrative example: urban wind field surrogate modeling

4.1. Problem statement

4.1.1. Dataset and input-output specification

The demonstration problem is neural surrogate modeling for pedestrian-level urban wind fields. The machine-learning subset used here follows Lu et al. (2025) and is derived from UrbanTALES (Nazarian et al., 2025); it contains 512 LES cases covering 224 idealized and 288 realistic urban configurations with four wind directions. The input to the surrogate is a two-dimensional representation of the building landscape, expressed as image-like contour fields on a 640×640 spatial grid. Depending on the input feature configuration, the input may include building height only (1 channel), building height with signed distance field (2 channels), or building height with signed distance field and surface normals (3 channels). The output is a single-channel two-dimensional pedestrian-level wind-speed field on the same spatial grid. The surrogate therefore learns a dense field-to-field mapping from urban form to the corresponding wind environment.

This setting is well suited to an automated research pipeline because it has a clear engineering objective, a reproducible data and evaluation contract, expensive but automatable training runs, and a large architecture-design space in which incremental scientific decisions can be tested systematically.

4.1.2. Reproduced baseline: Lu et al. (2025)

Lu et al. (2025) is used as the reproducible initial reference for the task. Its workflow provides the starting data representation, baseline architecture family, and evaluation metrics. The present study has fully reproduced the Lu baseline using a 7-level U-Net candidate with a channel progression of 16, 32, 64, 128, 256, 512, 1024 and approximately 34.6 million trainable parameters. The reproduced baseline uses the Adam optimizer with a fixed learning rate of 0.001, a batch size of 16, and a masked L1 loss computed only over non-building pixels. The baseline was trained for 1000 epochs on each of 20 random splits (410 training, 102 test per split) using the UrbanTALES dataset.

The reproduced baseline achieves a global R^2 of 0.687 (seed 1), with per-case median R^2 of 0.702, consistent with the original values reported by Lu et al. (2025) ($R^2 = 0.682$ global). Across 20 seeds, the reproduced model wins all 20 seed comparisons. Performance differs substantially between idealized configurations (per-case median $R^2 = 0.906$) and realistic configurations (per-case median $R^2 = 0.569$ for seed 1), confirming that realistic urban geometries with complex building arrangements represent the primary modeling challenge. Figure 4 shows the reproduced Lu-style U-Net architecture used as the initial reference architecture. Representative prediction results are shown in Figure 5, using the best model candidate discovered by the automated pipeline in Section 4.3.

4.1.3. Virtual human-scientist contrast: Snaiki et al. (2026)

Snaiki et al. (2026) proposes a hierarchical deep-learning model for predicting pedestrian-level urban winds. It uses a pretrained Lu-style U-Net as a first-stage predictor and a Pix2Pix-style conditional generative adversarial network (cGAN) as a second-stage refiner. The generator uses an enhanced U-Net candidate with stepwise kernels (32, 32, 16, 16, 8, 8, 8, 4), and the discriminator uses a PatchGAN architecture. Training uses a pixel-wise L1 loss with weight $\lambda = 100$ combined with an adversarial binary cross-entropy loss, a generator learning rate of 2×10^{-4}, a discriminator learning rate of 1×10^{-4}, a batch size of 4, and a maximum of 500 epochs with early stopping (patience 20). The reported best epoch is 146.

The headline reported metrics are validation median $R^2 = 0.945$, RMSE of 0.056 m/s, and MAE of 0.034 m/s. These values are substantially better than the current automated one-stage search results. However, several reliability caveats limit the use of Snaiki et al. (2026) as a strict benchmark: (1) the second-stage 90/10 split may overlap with the upstream first-stage training set (80/20 split), creating a structural data leakage concern; (2) the validation set is used for checkpoint selection with no independent held-out test set; (3) only a single run is reported with

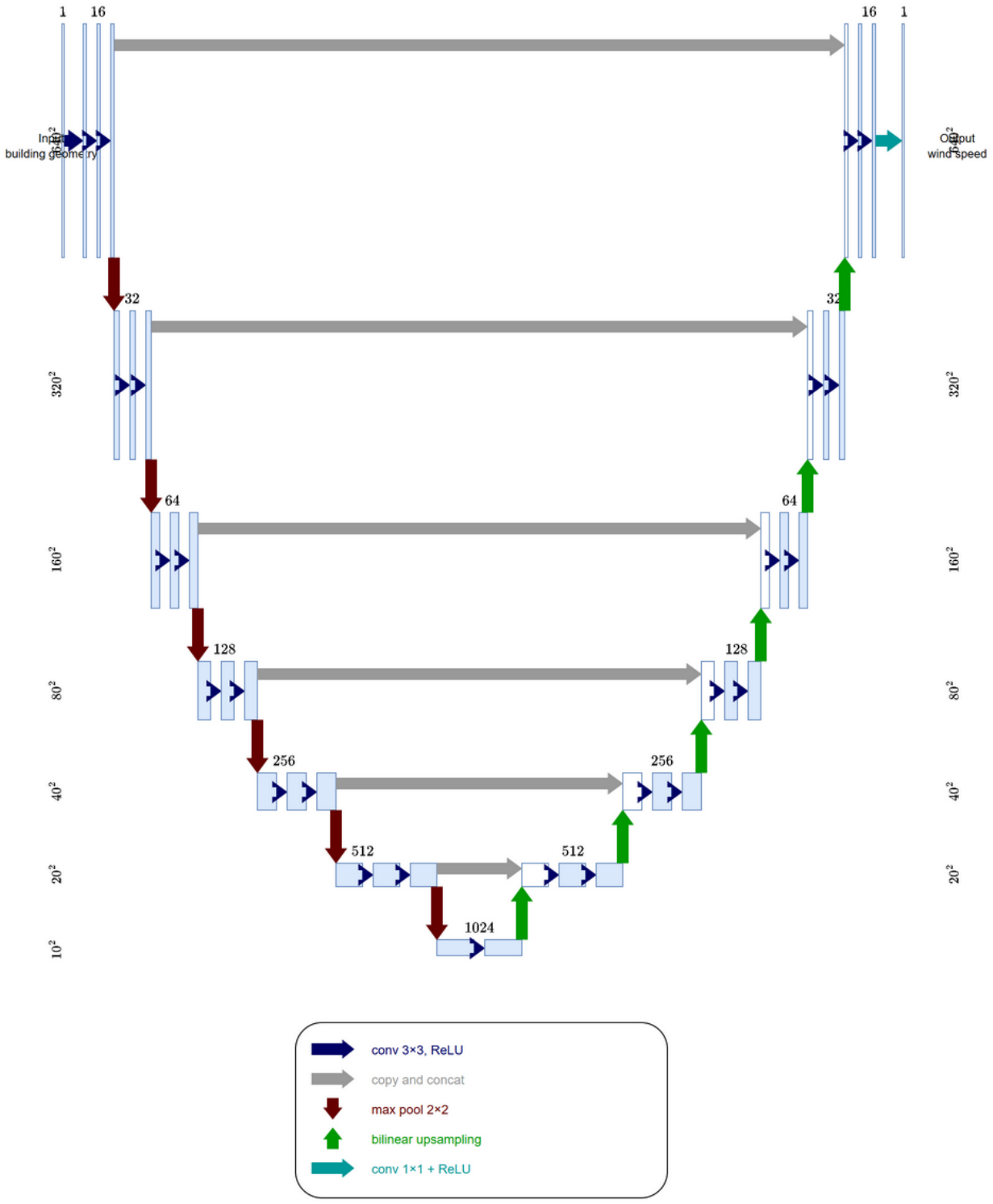


Figure A1. Diagram of U-net architecture used in this study.

Figure 4: Reproduced Lu et al. (2025) Figure A1-style U-Net architecture used as the initial reference model. The diagram was generated during the baseline reproduction and is included here to document the baseline architecture from which the automated search departs.

no seed averaging; and (4) the reported panels exclude cases with negative baseline R^2, and no code, weights, or evaluation scripts are released. Snaiki et al. (2026) is therefore treated as a contextual virtual human-scientist contrast rather than as a strict numerical target. A more detailed discussion of benchmark fairness appears in Section 5.7.

4.1.4. Data splits and evaluation protocol

The present campaigns use fixed train, validation, and holdout partitions within each audited protocol. The training set contains 410 cases, following the seed convention of Lu et al. (2025), the validation set contains 55 cases, and the holdout set contains 47 cases, summing to 512 total cases. During architecture search, the primary controller metric is the validation per-case median R^2. Holdout evaluation is reserved for selected candidates or posthoc audits, so it does not directly steer routine proposal generation. A total-evaluation summary (total102, combining 55 validation and 47 holdout cases) is used where split-matched posthoc evidence is available. Throughout the training campaign (seed 1 is used), the baseline validation median R^2 is 0.669, the holdout median R^2 is 0.750, and the total102 median R^2 is 0.698. These campaign-baseline values should be distinguished from the per-case median R^2 of 0.702 reported above for the reproduced Lu baseline: the value of 0.702 refers to the 1000-epoch reproduction evaluated on all 102 non-training cases of seed 1, whereas the campaign baseline is the reproduced U-Net retrained for 200 epochs, matching the full-training budget of the search, and evaluated on the 55-case validation and 47-case holdout partitions of the same 102 cases. All baseline comparisons in the campaigns below use the 200-epoch campaign baseline. These partitions were not constructed by grouping cases that share the same building geometry, so dependence between training and evaluation cases cannot be excluded; such dependence is a recognized source of over-optimistic evaluation in machine-learning-based science (Kapoor and Narayanan, 2023). All scores reported below should therefore be read as performance under this locked split, not as evidence of generalization to unseen building geometries.

The training protocol separates feasibility, model selection, and posthoc auditing. Feasibility is established by smoke tests and parameter-cap screening. Routine campaign ranking is based on validation per-case median R^2, because it can be computed consistently for all completed candidates without consuming the holdout set as an everyday planning signal. Holdout and total-evaluation summaries (combining validation and holdout cases) are reserved for selected candidates, multi-seed audits (3 seeds in this study), and cross-checks of robustness. This separation is central to the interpretation of results: a model that is best on a validation ranking, a model that is best on selected holdout evidence, and a model that is best on a three-seed total-evaluation summary are related but not identical claims.

4.2. Application modes

The same automated research logic is examined through three application modes that differ in how strongly the planner uses prior results before proposing the next candidates. All three modes

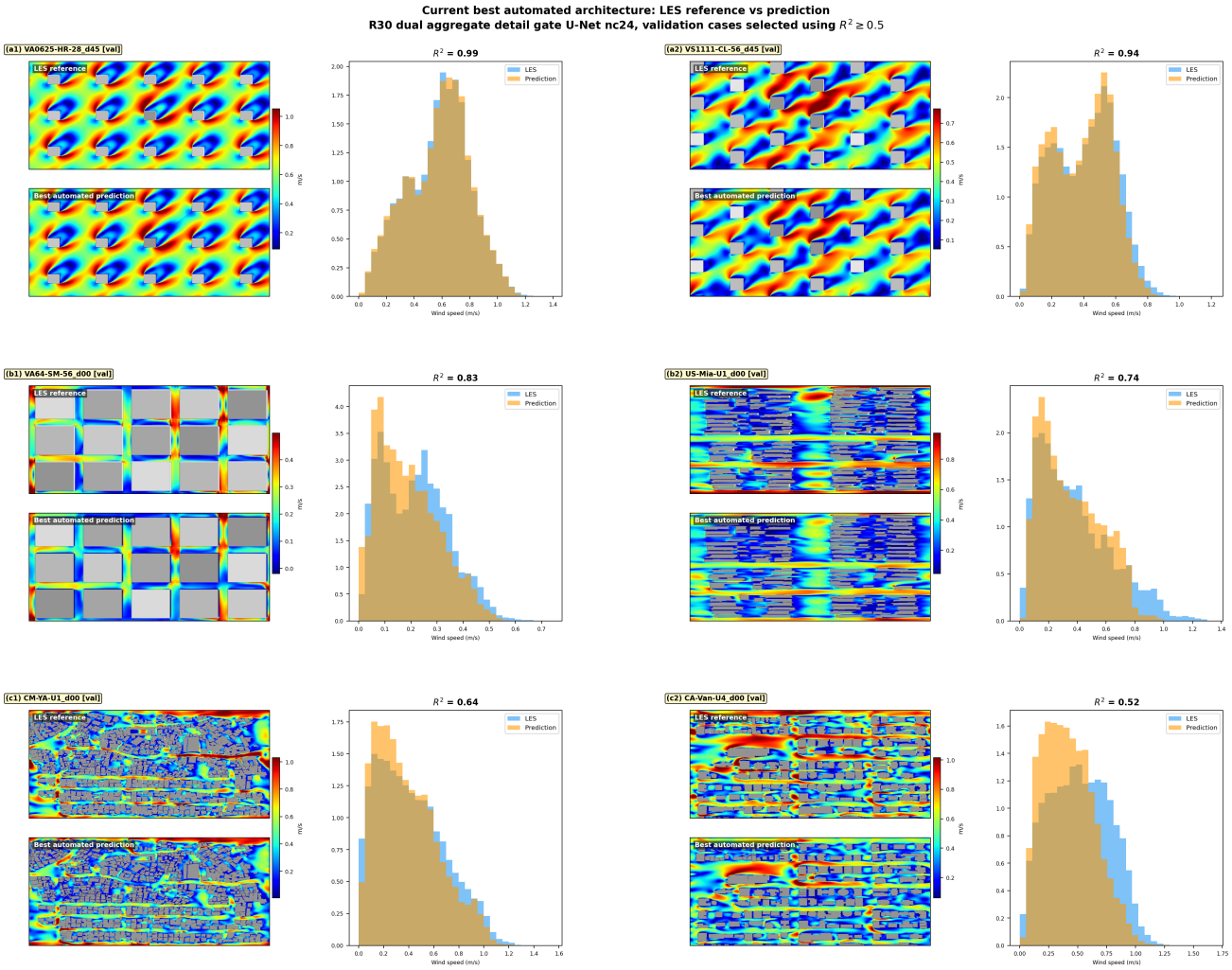


Figure 5: Lu et al. (2025) Figure 7-style qualitative comparison redrawn with the best model design discovered by the sequential mode in Section 4.3, R30 dual aggregate detail gate U-Net nc24. Six validation cases with $R^2 \geq 0.5$ are selected from the seed-1 evaluation to span the performance range. Each panel compares the LES reference wind-speed field with the model prediction and the corresponding wind-speed distribution; buildings are overlaid in gray.

share the same general phases of the automated research pipeline. Figure 6 provides a schematic comparison.

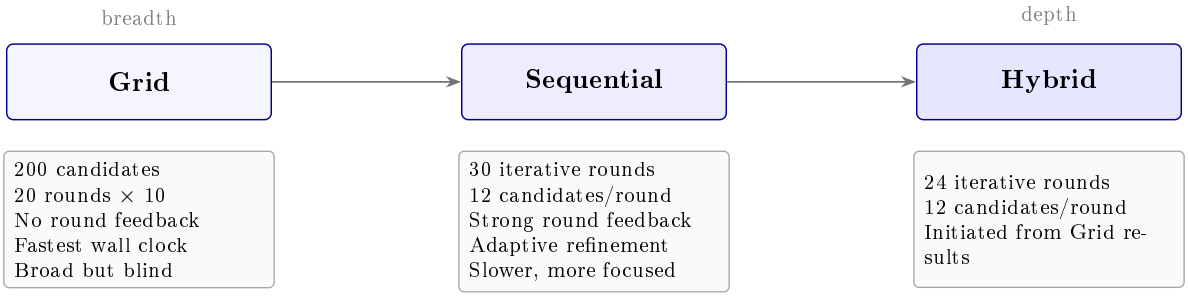


Figure 6: Three application modes of the automated research pipeline. The grid mode emphasizes breadth and speed; the sequential mode emphasizes adaptive learning from prior evidence; the hybrid mode combines an initial broad exploration phase with subsequent sequential refinement.

4.2.1. Grid mode

Grid mode emphasizes breadth and speed. In the grid-mode campaign, the planner produced 20 rounds with 10 source candidates per round at the beginning of the campaign, yielding approximately 200 source candidates. Eligible candidates were processed under a common 200-epoch benchmark protocol after smoke-stage checks and formal parameter-cap screening, with limits of 150 million parameters and 40 GB VRAM. The benchmark protocol separates recipe search, which identifies effective training recipes (particularly hyper parameters as listed in Table A.8) using a representative architecture, from architecture evaluation, which applies the top recipes uniformly to all candidate architectures. This mode can evaluate many architectures quickly because candidate generation is only weakly dependent on the immediately preceding results. It is the fastest in wall-clock organization, but it does not fully exploit round-by-round evidence. In the terminology of neural architecture search, which distinguishes the search space, the search strategy, and the performance-estimation strategy (Elsken et al., 2019), the grid mode fixes the search space to predefined architecture and hyperparameter tables and assigns every eligible candidate the same full-training budget; adaptive multi-fidelity schedulers such as Hyperband (Li et al., 2018), which allocate training budget adaptively and stop weak configurations early, were not used.

4.2.2. Sequential mode

Sequential mode emphasizes adaptive learning. In the sequential-mode campaign, the planner and reviewer operated over 30 full-training rounds (R2–R31; R31 was executed after the predefined Round 30 stopping point), typically with 12 candidates per round. Later proposals were informed by accumulated validation results, smoke failures, family-level summaries, and reviewer diagnoses. This mode resembles an advanced human research loop, in which each round updates the next

experimental plan. It is slower than the grid mode because each proposal batch depends on completed evidence, but it can refine architecture families, retire repeated failures, and focus computation on increasingly plausible design directions.

4.2.3. Hybrid mode

Hybrid mode is intended to combine the behaviors of grid and sequential modes. In this mode, the protocol starts from the grid-mode results, which provide an initial exploration phase of approximately 200 candidates, and then transitions to sequential refinement as round-level evidence accumulates. The hybrid-mode campaign uses 12 experiments per round and follows the same locked data split and evaluation contract as the other modes. It does not inherit any experimental results from the sequential-mode campaign. Instead, it starts from a clean state, with only operational preferences and constraint rules carried forward from the grid mode.

4.3. Results

4.3.1. Evaluation metrics and split discipline

The results reported in this section use three levels of evaluation evidence, which are kept explicitly labeled throughout. Let U_i denote the reference wind speed at unmasked spatial point i, $\hat{U}_i$ the model prediction, $\bar{U}$ the mean reference wind speed over the same points, and N the number of evaluated points. Two types of metrics are calculated:

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(U_i - \hat{U}_i)^2}{\sum_{i=1}^{N}(U_i - \bar{U})^2}\,. \tag{1}$$

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|U_i - \hat{U}_i|\,. \tag{2}$$

MAE is used during training to guide model optimization. In addition, the latter result tables report per-case R^2, which is used as the primary campaign-control metric. Normalized MAE and standard-deviation error were considered as possible auxiliary diagnostics, but they are not reported due to space constraints. Per-case values are aggregated across the case set using either the median or the mean, depending on the reported statistic. In the round-trajectory figures, candidates with median $R^2 < 0.5$ are omitted so that failed or near-failed training runs do not compress the informative vertical range.

Validation median R^2 is the primary campaign-control metric. It is computed over the 55-case validation split for each completed candidate and is used for routine round-by-round ranking and planner feedback. Holdout median R^2 is computed over the 47-case holdout split and is available only for candidates specifically selected for holdout evaluation; it does not steer routine proposal generation. Total102 median R^2 combines the 55 validation and 47 holdout cases (102 total) and is available in posthoc three-seed summaries for selected candidates. The checkpoint selection is

performed on the stratified val55 split only; total102 is not used to choose checkpoints and is used only for final reporting and ranking.

Three-seed evaluation trains the same candidate with three different random seeds and reports the mean, median, minimum, and maximum of the per-seed total102 metrics. This evaluation is used to assess robustness and to separate genuine architecture improvements from seed-dependent fluctuations. A 20-seed evaluation of the global top configuration, which would provide a more demanding seed-sensitivity analysis, was not performed in this study and is left for future work.

Parameter count is reported for all candidates because the automated pipeline explores a wide range of model sizes.

4.3.2. Grid-mode results

In the grid-mode campaign, approximately 200 source candidates were generated across 20 rounds with 10 candidates per round. Of these, 172 were present in the validation-ranked benchmark table and 28 were formally parameter-cap skipped. The ranked parameter range spans from 206,337 to 144,933,361 parameters. The benchmark evidence contains 172 validation rows across 34 distinct architecture labels.

Table 1 reports the top 12 validation-ranked candidates. The best validation-ranked candidate is the R3 U-Net/SDF 7-level model, which reaches validation median $R^2 = 0.735$ with approximately 125.8 million parameters. A Fourier U-Net candidate reaches validation median $R^2 = 0.721$ with approximately 34.2 million parameters, demonstrating that competitive performance can be achieved at lower parameter counts.

Selected holdout evidence changes the interpretation but does not invalidate the validation ranking. As shown by Figure 7, among the 10 top validation-ranked candidates selected for holdout evaluation, the round-18 U-Net/SDF 7-level variant reaches holdout median $R^2 = 0.775$, followed by other variants at 0.766 (round 17), 0.762 (round 19), 0.752 (round 9, Fourier U-Net), and 0.750 (round 3, U-Net/SDF 7-level). The top rank in validation does not always guarantee the best holdout performance. The grid-mode result is best interpreted as a broad discovery pass that rapidly identified a high-performing U-Net/SDF family, while still requiring posthoc audits for robustness and parameter efficiency.

The model-family analysis across the 172 ranked rows reveals that the U-Net/SDF 7-level family achieved the best individual and median family performance, the Fourier U-Net family was the second-best family, and the Dilated U-Net family achieved a best validation median R^2 of 0.704 with only 14.8 million parameters. Using the current baseline, 45 of 172 ranked rows exceed the validation median R^2 baseline, and every round produced at least one ranked candidate above the validation baseline.

4.3.3. Sequential results

The sequential-mode campaign provides the clearest evidence for adaptive multi-round improvement because later proposals were generated after reviewer summaries and accumulated fail-

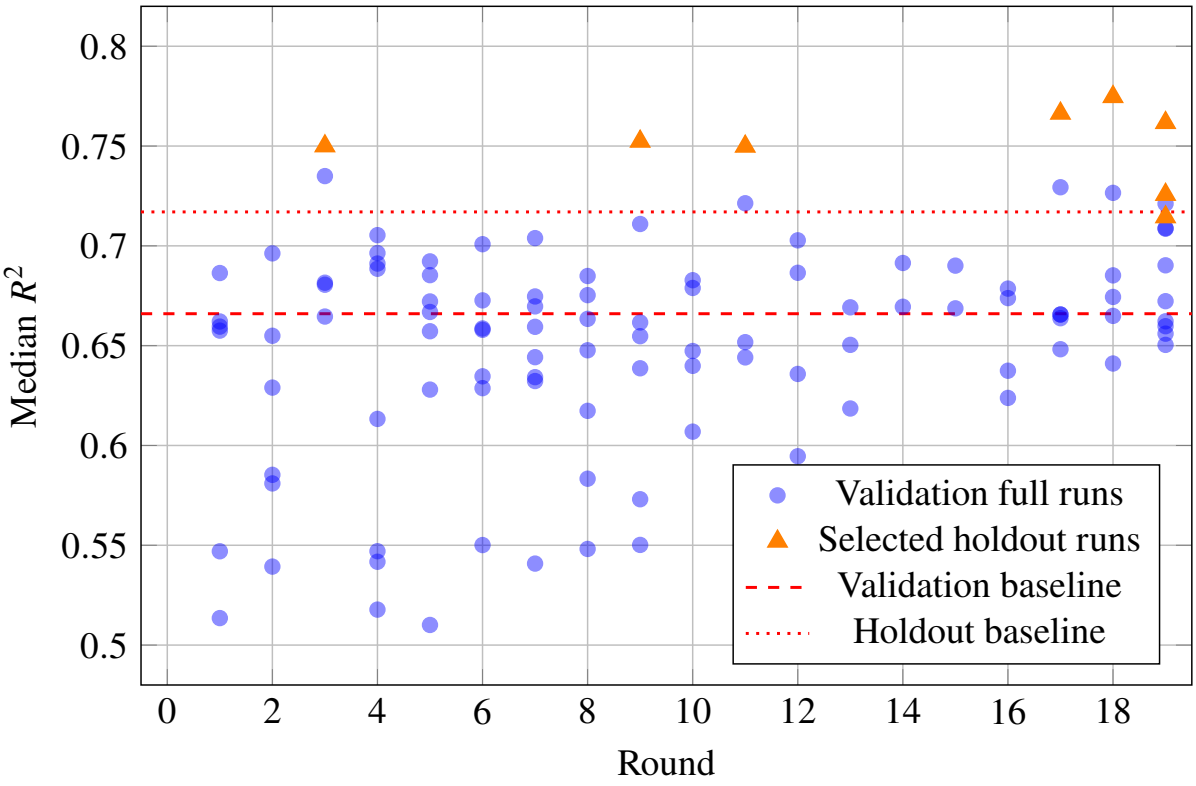


Figure 7: Grid-mode validation and selected holdout median R^2 by round. Blue markers show validation-ranked benchmark records; orange markers show selected candidates evaluated on the 47-case holdout split. Dashed and dotted red lines mark the split-matched validation and holdout baselines, respectively. Candidates with median $R^2 < 0.5$ are excluded.

Table 1: Top 12 grid-mode candidates ranked by validation median R^2 in the grid-mode campaign. The baseline validation median R^2 is 0.666. All results are single-seed 200-epoch full-training runs.

Rank	Round	Candidate	Med. R^2	Med. MAE	Params (M)
1	3	U-Net/SDF 7-level	0.735	0.085	125.8
2	17	U-Net/SDF 7-level	0.729	0.087	125.8
3	18	U-Net/SDF 7-level	0.727	0.085	125.8
4	11	Fourier U-Net	0.721	0.083	34.2
5	19	U-Net/SDF 7-level	0.721	0.084	125.8
6	9	Fourier U-Net	0.711	0.087	34.2
7	19	U-Net/SDF 7-level	0.709	0.088	125.8
8	19	UNetV2 baseline	0.709	0.087	34.6
9	4	CNOv2	0.705	0.087	144.9
10	7	Dilated U-Net	0.704	0.089	14.8
11	12	Multiscale convolutional U-Net	0.703	0.086	75.2
12	6	CNOv2	0.701	0.090	144.9

ure/performance evidence. Over 30 numbered rounds, the workflow produced 319 full-training records, of which 246 exceeded the split-matched baseline on the 55-case validation split (validation median $R^2 = 0.669$).

Table 2 reports the top 10 validation-ranked candidates. The best validation result is the R19 UNetV2 nc48, a large model with 311.41 million parameters reaching validation median $R^2 = 0.749$. However, several compact models from later rounds achieve nearly equivalent validation performance. The R31 dual aggregate detail gate U-Net nc24 reaches 0.747 with 7.58 million parameters, and the R31 dual aggregate detail gate U-Net nc22 reaches 0.744 with approximately 6.5 million parameters.

On the 47-case holdout split, the best selected candidate, the R31 dual aggregate detail gate U-Net nc22, achieves a holdout median R^2 of 0.788, exceeding the baseline holdout value of 0.750 by +0.039. The R31 hypercoord microdecoder U-Net nc24 reaches holdout $R^2 = 0.787$ with 5.98 million parameters. Table 3 presents the selected holdout evaluation results.

The three-seed posthoc evaluation provides the most robust evidence for the sequential campaign. Table 4 reports the top candidates ranked by three-seed total102 mean R^2. The baseline three-seed total102 mean is 0.728 with a standard deviation of 0.015. The R30 dual aggregate detail gate U-Net nc24 achieves the highest three-seed total102 mean of 0.785, corresponding to an absolute gain of +0.057 and a 21.0% reduction in the residual gap $1 - R^2$, with 7.58 million parameters. The R31 dual aggregate detail gate U-Net nc22 achieves a three-seed mean of 0.782 with the highest single-seed maximum of 0.822. The R30 tensorized spectral adapter U-Net nc24

Table 2: Top 10 sequential-mode candidates ranked by validation median R^2 in the sequential-mode campaign. The baseline validation median R^2 is 0.669. All results are single-seed 200-epoch full-training runs.

Rank	Round	Candidate	Val. med. R^2	Δ	Val. MAE	Params (M)
1	19	UNetV2 nc48	0.749	+0.080	0.086	311.41
2	31	Dual aggregate detail gate U-Net nc24	0.747	+0.078	0.085	7.58
3	31	Dual aggregate detail gate U-Net nc22	0.744	+0.075	0.080	6.5
4	28	Tensorized spectral adapter U-Net nc24	0.742	+0.072	0.084	6.23
5	29	Wavelet-refine U-Net nc32	0.742	+0.072	0.084	9.38
6	30	Tensorized spectral adapter U-Net nc24	0.741	+0.072	0.086	6.23
7	30	Dual aggregate detail gate U-Net nc24	0.741	+0.072	0.084	7.58
8	31	Hypercoord microdecoder U-Net nc24	0.741	+0.072	0.082	5.98
9	31	Omniscan skip mixer U-Net nc24	0.741	+0.072	0.082	6.2
10	13	SAC U-Net nc32	0.739	+0.069	0.089	237.69

Table 3: Selected sequential-mode candidates evaluated on the 47-case holdout split, ranked by holdout median R^2. The baseline holdout median R^2 is 0.750.

Rank	Round	Candidate	Params (M)	Val. R^2	Hold. R^2	Δ	Hold. MAE
1	31	Dual aggregate detail gate U-Net nc22	$\sim$6.5	0.744	0.788	+0.039	0.065
2	31	Hypercoord microdecoder U-Net nc24	5.98	0.741	0.787	+0.037	0.071
3	30	Hypercoord microdecoder U-Net nc24	5.98	0.738	0.783	+0.034	0.071
4	30	Wavelet-refine U-Net nc36 (retry2)	11.87	0.729	0.781	+0.032	0.069
5	22	CNO nc40	402.98	0.730	0.778	+0.029	0.072
6	30	Tensorized spectral adapter U-Net nc24	6.23	0.741	0.777	+0.027	0.070
7	19	UNetV2 nc48 (best)	311.41	0.749	0.776	+0.027	0.073
8	23	CNO nc44	1950.75	0.735	0.770	+0.020	0.076

achieves a three-seed mean of 0.779 with median of 0.781.

Table 4: Three-seed posthoc evaluation of top sequential-mode candidates, ranked by total102 mean R^2. The baseline three-seed total102 mean R^2 is 0.728. Δ is the gain over the baseline mean.

Rank	Candidate	Mean	Median	Min	Max	Δ	Gap red. (%)
1	R30 dual aggregate detail gate U-Net nc24	0.785	0.775	0.767	0.814	+0.057	21.0
2	R31 dual aggregate detail gate U-Net nc22	0.782	0.763	0.761	0.822	+0.054	19.7
3	R30 tensorized spectral adapter U-Net nc24	0.779	0.781	0.760	0.797	+0.051	18.7
4	R27 boundary cross-attention U-Net nc24	0.777	0.769	0.767	0.795	+0.049	17.9
5	R30 shape-basis U-Net nc24	0.777	0.777	0.763	0.791	+0.049	17.9
6	R28 tensorized spectral adapter U-Net nc24	0.774	0.779	0.761	0.782	+0.046	16.7
7	R29 wavelet-refine U-Net nc32	0.771	0.774	0.749	0.790	+0.042	15.6
8	R31 omniscan skip mixer U-Net nc24	0.769	0.754	0.750	0.802	+0.040	14.8
–	Baseline UNetV2 nc16	0.728	0.728	0.710	0.747	0.000	0.0

These results indicate that the sequential process did not simply scale parameter count to achieve improvement. The top sequential candidates are much smaller than the 125.8 million parameter best validation-ranked grid mode seven-level U-Net and far smaller than the 311 to 1951 million parameter large UNetV2/CNO entries retained in the sequential-mode posthoc evidence. The R31 dual aggregate detail gate U-Net nc22 achieves the best single-split holdout median $R^2 = 0.788$ with approximately 6.5 million parameters, exceeding the 311 million parameter UNetV2 (holdout $R^2 = 0.776$) at roughly 1/48 the parameter count.

The three-seed evidence also exposes variance. For top sequential candidates, the three-seed total102 standard deviations are on the order of 0.009 to 0.021, while the baseline has a standard deviation of 0.015. This variance is large enough that final claims should not depend on a single lucky seed, but small enough that the best sequential family remains consistently stronger than the baseline.

4.3.4. Hybrid results

The hybrid-mode campaign is reported by using 280 completed seed-1 full-training records over 24 rounds on the locked 410/55/47 split. The best seed-1 validation result was the R17 anisotropic boundary hybrid U-Net, which reached validation median $R^2 = 0.7573$ with 4.67 million parameters; an R23 run of the same family reached 0.7563. The Top-12 seed-1 validation candidates were then evaluated on the 47-case seed-1 holdout split. The strongest seed-1 holdout entries were the R12 boundary-gated multiscale U-Net at holdout median $R^2 = 0.7942$, the R23 terrain local-attention U-Net at 0.7930, and the R12 anisotropic-kernel U-Net at 0.7925. A subsequent three-seed audit evaluated the selected hybrid candidates using seeds 2, 3, and 4. For each seed and run,

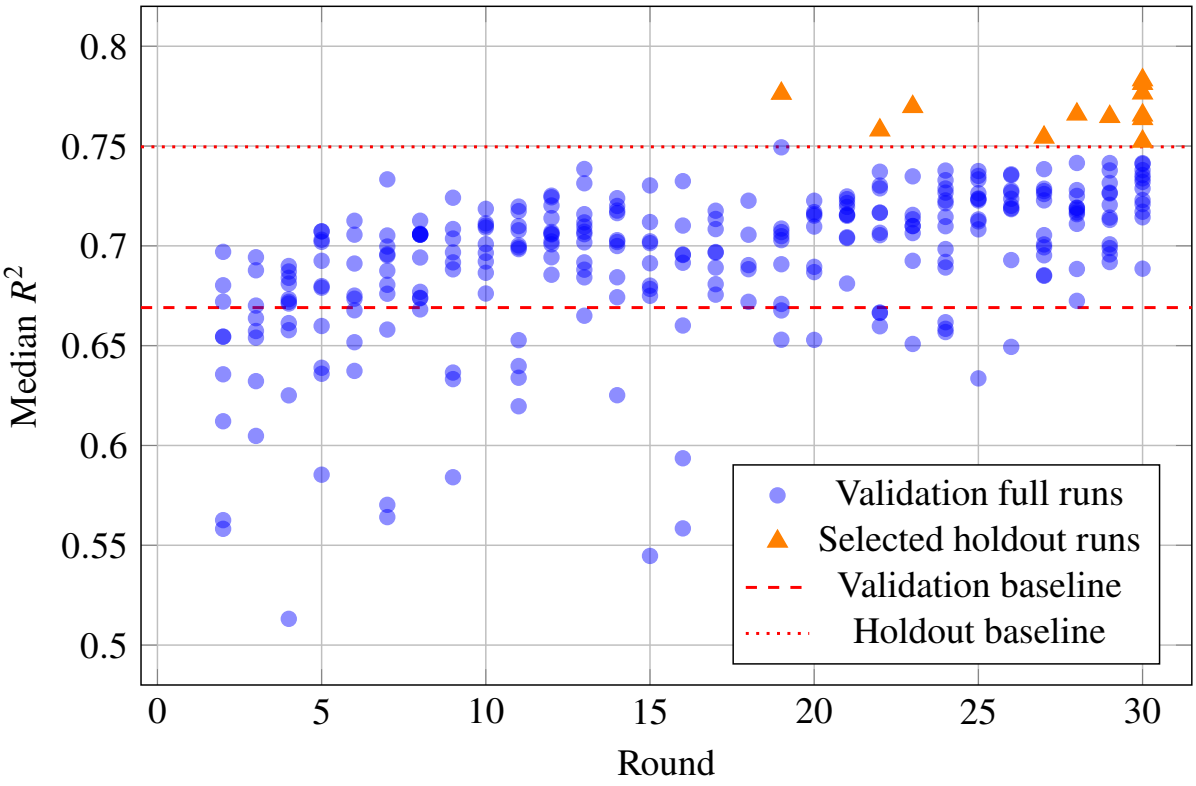


Figure 8: Sequential-mode validation and selected holdout median R^2 by round. Blue markers show all the full-training validation records; orange markers show selected candidates evaluated on the holdout split. Dashed and dotted red lines mark the split-matched epoch-200 validation and holdout baselines, respectively. Candidates with median $R^2 < 0.5$ are excluded.

the checkpoint was selected by corrected stratified val55 median R^2; the corresponding total102 median R^2 was then reported and used for the final ranking. Total102 was not used for checkpoint selection. Figure 9 shows the hybrid-mode validation and selected holdout trajectory; Tables 5 and 6 summarize the final-report evidence in the compact style used for the other campaign tables in this study.

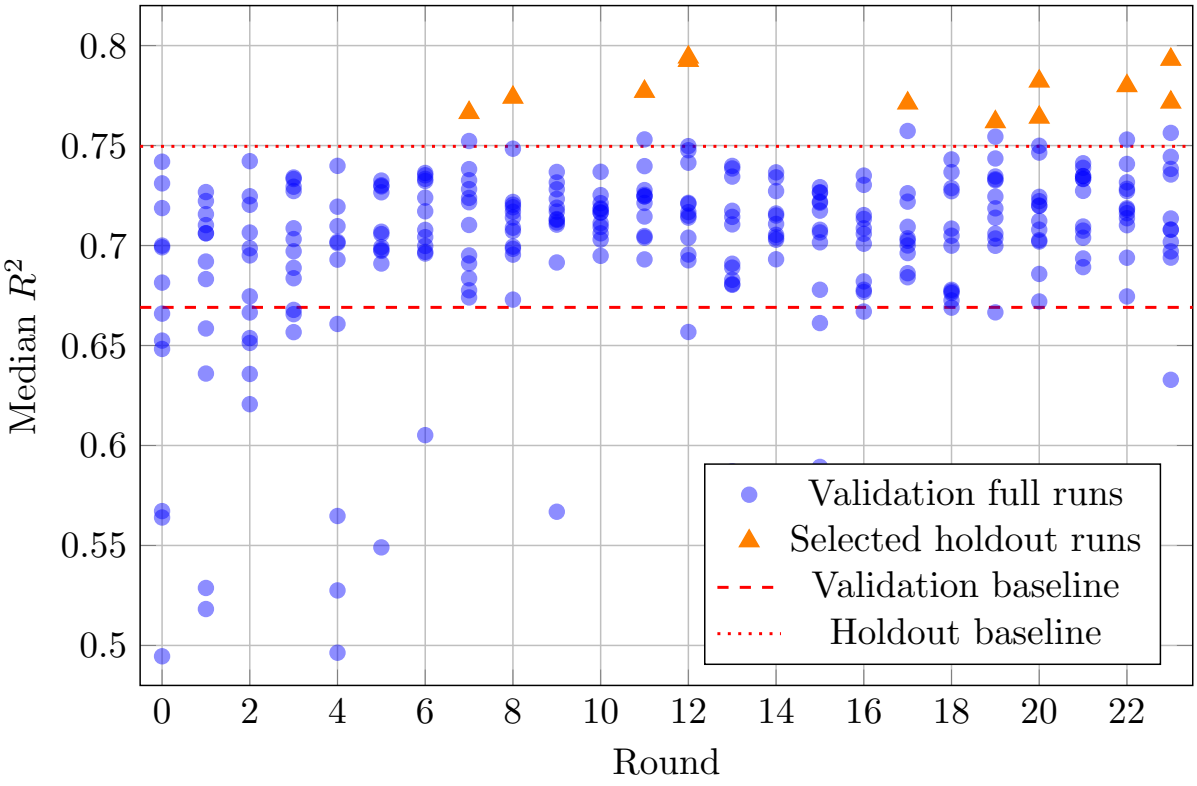


Figure 9: Hybrid-mode validation and selected holdout median R^2 trajectory. Blue markers show the 280 completed validation records; orange markers show the selected Top-12 candidates evaluated on the holdout split. Dashed and dotted red lines mark the split-matched validation and holdout baselines, respectively.

The three-seed posthoc ranking revises the interpretation that the seed-1 validation leaderboard would have suggested. The formal hybrid-mode winner is the Top6 R11 anisotropic-kernel U-Net, configured with $n_c = 20$, depth 6, and learning rate 4×10^{-4}, with mean total102 median $R^2 = 0.7946$. The Top3 R12 anisotropic-kernel U-Net with $n_c = 24$ and learning rate 3.5×10^{-4} ranks second at 0.7885, and the Top5 R22 terrain local-attention U-Net retry1 ranks third at 0.7878. The seed-1 holdout screen therefore acts as a selection filter rather than as a final ranking: it elevated candidates that were not the seed-1 validation leaders, but the final order is fixed only by the three-seed posthoc evidence. The holdout split also appears more discriminative than

Table 5: Hybrid-mode Top-12 validation candidates with their corresponding holdout screening ranks and median R^2 values. The table follows the same split-aware reporting style as the sequential mode; validation ranking was used for search control, while holdout ranking was used only for the selected Top-12 screen.

Val. rank	Hold. rank	Candidate	Params (M)	Val. med. R^2	Hold. med. R^2	lr
1	9	R17 anisotropic boundary hybrid U-Net nc20	4.67	0.7573	0.7712	0.0004
2	8	R23 anisotropic boundary hybrid U-Net nc20	4.67	0.7563	0.7716	0.0004
3	12	R19 hypernet residual U-Net nc20	5.67	0.7544	0.7618	0.0004
4	6	R11 anisotropic-kernel U-Net nc20	6.88	0.7531	0.7770	0.0004
5	5	R22 terrain local-attention U-Net nc20	6.81	0.7530	0.7799	0.0004
6	10	R7 TT-adapter decoder U-Net nc24	6.10	0.7523	0.7664	0.0005
7	4	R20 terrain local-attention U-Net nc20	6.81	0.7499	0.7821	0.0004
8	3	R12 anisotropic-kernel U-Net nc24	9.90	0.7497	0.7925	0.00035
9	7	R8 boundary-gated multiscale U-Net nc24	7.78	0.7484	0.7741	0.0004
10	1	R12 boundary-gated multiscale U-Net nc24	7.78	0.7477	0.7942	0.0003
11	11	R20 anisotropic boundary hybrid U-Net nc20	4.67	0.7465	0.7641	0.0004
12	2	R23 terrain local-attention U-Net nc20	6.81	0.7444	0.7930	0.0004

Table 6: Hybrid-mode three-seed posthoc evaluation, ranked by total102 mean R^2. The baseline three-seed total102 mean R^2 is 0.728. Δ is the gain over the baseline mean.

Rank	Candidate	Mean	Median	Min	Max	Δ	Gap red. (%)
1	Top6 R11 anisotropic-kernel U-Net nc20 lr4e-4	**0.795**	0.779	0.768	0.836	+0.067	24.6
2	Top3 R12 anisotropic-kernel U-Net nc24 lr3.5e-4	0.788	0.780	0.769	0.816	+0.060	22.1
3	Top5 R22 terrain local-attention U-Net retry1	0.788	0.783	0.778	0.803	+0.060	22.1
4	Top2 R23 terrain local-attention U-Net retry2	0.784	0.783	0.776	0.793	+0.056	20.6
5	Top1 R12 boundary-gated multiscale U-Net nc24 lr3e-4	0.781	0.768	0.763	0.811	+0.053	19.5
6	Top4 R20 terrain local-attention U-Net	0.779	0.777	0.761	0.798	+0.051	18.8
7	Top7 R08 boundary-gated multiscale U-Net lr4e-4	0.776	0.769	0.765	0.794	+0.048	17.6
—	Baseline UNetV2 nc16	0.728	0.728	0.710	0.747	0.000	0.0

the validation split for several validation-leading anisotropic boundary hybrid U-Net runs, which dropped from validation ranks 1–2 to holdout ranks 8–9.

4.3.5. Cross-mode comparison

Table 7 summarizes the current cross-mode evidence. The sequential-mode best audited row reaches three-seed total102 mean $R^2 = 0.785$. The grid-mode selected holdout rows reach holdout median R^2 values up to 0.775. The hybrid-mode's strongest row is the R11 anisotropic-kernel U-Net, with mean total102 median $R^2 = 0.7946$ and approximately 6.88 million parameters, which is the strongest row among all modes. This cross-mode comparison is not a controlled ablation of the search mode. All sequential-mode full-training runs (400 run directories) used height-only input, whereas 263 of the 321 hybrid-mode full-training run directories, including all seven candidates in the hybrid three-seed posthoc ranking, used height, SDF, and surface-normal inputs; several hybrid candidates (the R20–R23 terrain local-attention variants) additionally used light geometric augmentation, and the hybrid mode started from grid-mode prior knowledge. The differences between modes therefore cannot be attributed to the search mode alone.

Table 7: Cross-mode comparison of the three automated application modes and the external human-designed contrast. The grid mode is reported using holdout median R^2 values for seed 1; the sequential and hybrid modes are reported using three-seed total102 results; Snaiki et al. (2026) is included only as contextual external evidence, not as a same-protocol ranking row.

Mode	Search logic	Best candidate	R^2	Interpretation
Grid	one broad batch	R18 U-Net/SDF 7-level	0.775 (holdout)	broad discovery, large model
Sequential	adaptive rounds	R30 dual aggregate detail gate U-Net nc24	0.785	compact and robust posthoc winner
Hybrid	grid-informed adaptive rounds	R11 anisotropic-kernel U-Net nc20	0.7946	strongest audited automated result
Snaiki et al. (2026)	human-designed two-stage refinement	predictor-refiner system	0.945	contextual target, not same-protocol comparison

The cross-mode pattern supports three practical observations. First, grid search is operationally fast and broad, but its best current candidates are large, with 125.8 million parameters for the top validation and holdout U-Net/SDF entries. Moreover, this mode is comparatively blind to round-by-round evidence, and its best validation results are not as strong as those obtained by the sequential mode. Second, sequential search produces more refined and parameter-efficient candidates, with approximately 6 to 8 million parameters for the top three-seed entries, because the planner and reviewer exploit accumulated prior evidence. During this mode, the performance improves gradually and consistently, as shown in Figure 8. Third, hybrid search can recover the best results by combining grid-based exploration with subsequent sequential refinement. Compared with Figure 8, the performance increase is less pronounced in Figure 9. However, most candidates in Figure 9 outperform the baseline in terms of validation performance. This is most likely because the planner effectively learns prior knowledge from the grid-mode results and avoids deficiencies discovered from the low-performing candidates. After 12 rounds, the hybrid mode had already revealed its best-performing candidate, which is also the best candidate among all modes.

4.3.6. Representative high-performing architecture details

The pooled three-seed total102 ranking suggests three architecturally distinct high-performing configurations to illustrate the range of mechanisms discovered by the automated pipeline. The full ranking, including both anisotropic-kernel variants, is preserved in Tables 6 and 5.

The three selected representatives are: (a) the R11 anisotropic-kernel U-Net (hybrid rank 1, mean total102 $R^2 = 0.7946$), (b) the R22 terrain local-attention U-Net retry1 (hybrid rank 3, 0.7878), and (c) the R30 dual aggregate detail gate U-Net nc24 (sequential-mode rank 1, 0.785). These three configurations differ in their core mechanism while sharing the same U-Net encoder-decoder backbone with reflection padding, skip concatenation, and bilinear upsampling. The first two hybrid-mode winners; the third is the strongest sequential-mode three-seed entry and is included to represent a structurally distinct high-performing design from a different campaign mode. The descriptions below are drawn from the active generated model files and full-training configuration files; modules outside the source are not inferred or extrapolated. Figure 10 provides a source-visible schematic with detail panels that show the internal data flow of each distinctive module.

(a) R11 anisotropic-kernel U-Net (rank 1, $R^2 = 0.7946$).. This candidate uses $n_c = 20$, depth 6, learning rate 4×10^{-4}, masked L1 gradient loss, three-channel input (height, SDF, surface normals), and approximately 6.88 million parameters. The U-Net backbone uses stride-2 convolutional downsampling, bilinear upsampling, and group normalization with SiLU activations. Its distinctive block projects features through a reflected 3×3 convolution, then applies parallel anisotropic reflected convolutions with 7×3 (vertical) and 3×7 (horizontal) kernels, concatenates the two directional responses, mixes them with a 1×1 convolution, applies a final 3×3 convolution, and adds a residual path. NaN inputs are masked to zero before prediction and reinstated outside the valid output mask. The anisotropic kernels provide directional sensitivity that can capture orientation-dependent flow features in the wind field without requiring full self-attention.

(b) R22 terrain local-attention U-Net (rank 3, $R^2 = 0.7878$).. This candidate uses $n_c = 20$, depth 6, learning rate 4×10^{-4}, masked L1 gradient loss, three-channel input with light geometric augmentation, and approximately 6.81 million parameters. Two learned 1×1 adapters split the input into a feature stream and a single-channel terrain signal. The encoder uses dual residual blocks per scale with average-pooling downsampling, and the decoder uses bilinear upsampling with dual residual blocks after skip concatenation. At each encoder scale, a terrain local-attention module multiplies features by a residual gate of the form $1 + g_{\text{local}}(x)\, g_{\text{terrain}}(t)$, where the local branch applies a depthwise reflected 3×3 convolution followed by 1×1 projections and a sigmoid, and the terrain branch passes the resized terrain signal through a reflected convolution and a sigmoid. The terrain conditioning allows each scale to modulate its representation based on the building-height field, providing an explicit link between urban morphology and learned features.

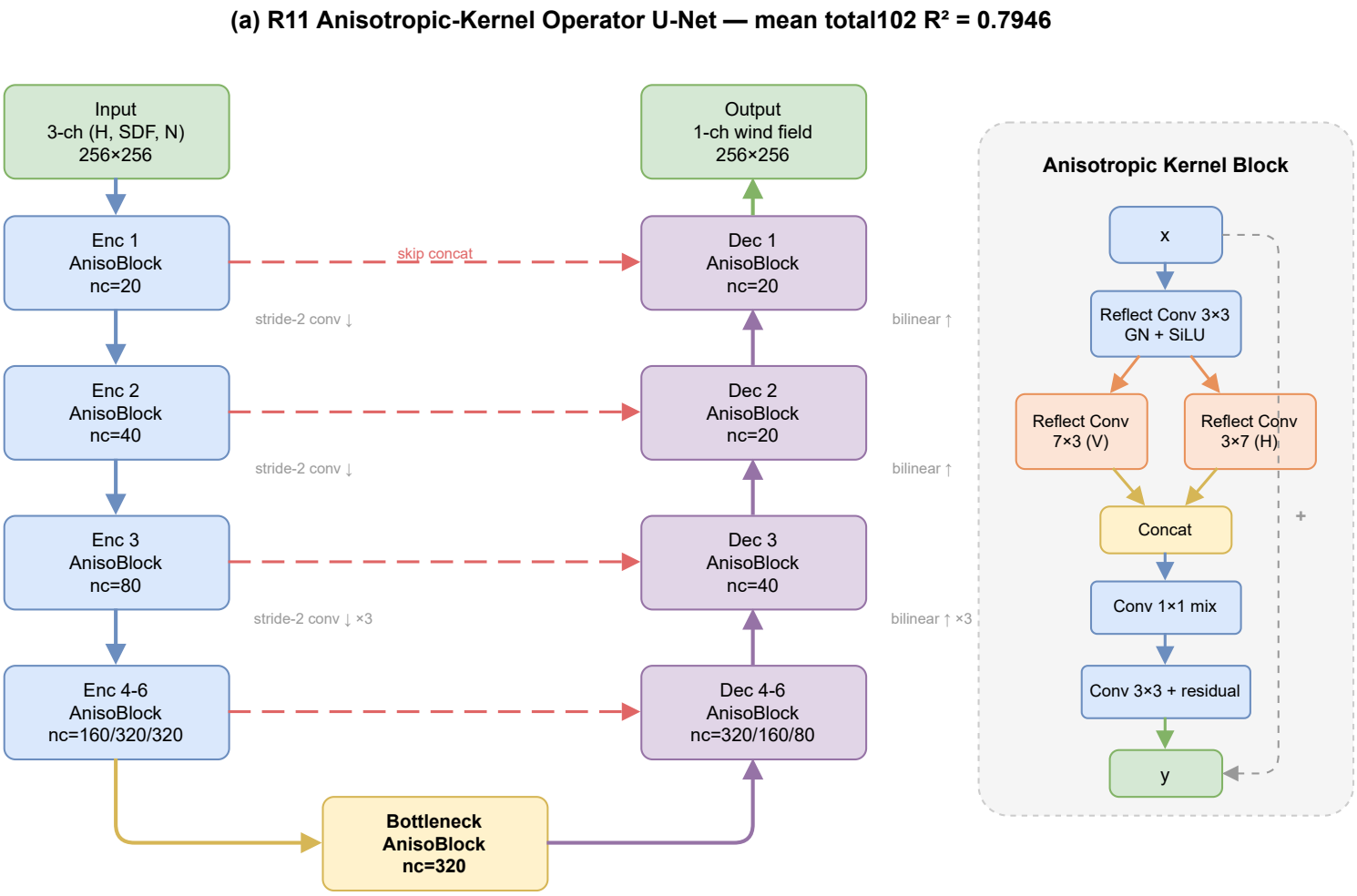


Figure 10: Source-visible schematic of three representative high-performing architectures selected for structural diversity from the corrected hybrid-mode and sequential-mode three-seed rankings. (a) The anisotropic-kernel U-Net uses parallel 7×3 and 3×7 reflected convolutions to capture directional sensitivity.

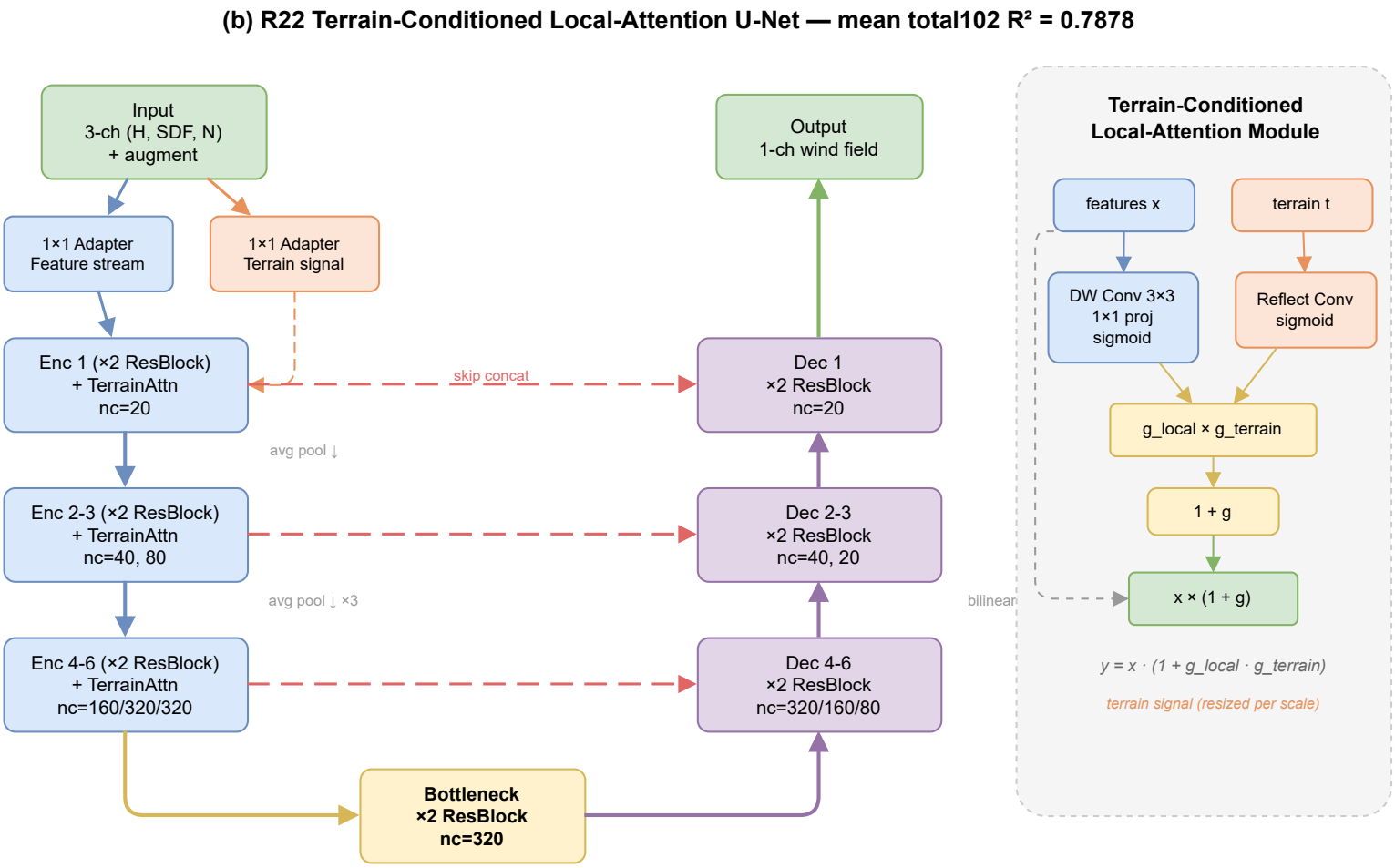


Figure 10: Source-visible schematic of representative high-performing architectures, continued. (b) The terrain local-attention U-Net multiplies features by a residual gate derived from a learned terrain signal and a depthwise local-attention branch.

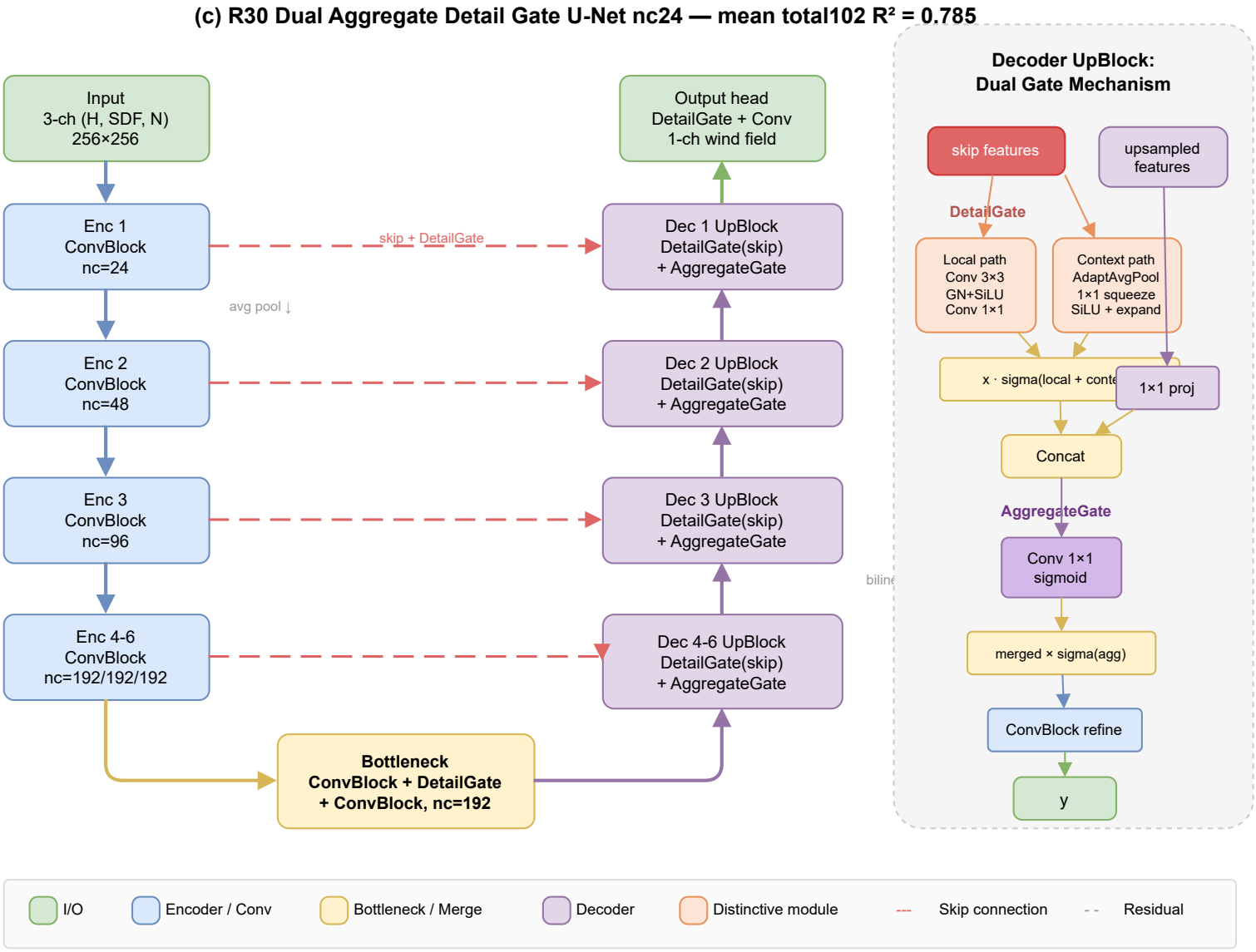


Figure 10: Source-visible schematic of representative high-performing architectures, continued. (c) The dual aggregate detail gate U-Net applies a two-path DetailGate to skip connections and an AggregateGate to the merged decoder features. All three models share the U-Net backbone with reflection padding, skip concatenation, and bilinear upsampling; the largest structural differences are confined to the highlighted modules.

(c) R30 dual aggregate detail gate U-Net nc24 (Table 4 rank 1, $R^2 = 0.785$).. This candidate uses $n_c = 24$, depth 6, learning rate 3×10^{-4}, masked L1 gradient loss, height-only input, and approximately 7.58 million parameters. The channel sequence is $[24, 48, 96, 192, 192, 192]$ (exponential growth capped at $8\times$ base width). The encoder uses single residual ConvBlocks (reflected 3×3 convolutions, group normalization, SiLU activations, residual skip) with average-pooling downsampling. The bottleneck applies a ConvBlock, a DetailGate, and a second ConvBlock in sequence. The distinctive mechanism is a two-level gating strategy in the decoder UpBlock. First, a DetailGate is applied to the skip connection: the gate combines a local path (reflected 3×3 convolution, group normalization, SiLU, 1×1 convolution) with a context path (adaptive average pooling, 1×1 squeeze, SiLU, 1×1 expansion), producing a multiplicative gate $x \cdot \sigma(\text{local} + \text{context})$. Second, the gated skip features are concatenated with the 1×1-projected upsampled features, and an AggregateGate (1×1 convolution followed by sigmoid) modulates the merged representation before a ConvBlock refines it. The output head applies a final DetailGate, a reflected convolution with group normalization and SiLU, and a 1×1 projection. NaN inputs are masked to zero before prediction and reinstated outside the valid output mask. The dual gating design separates detail enhancement on skip connections from fusion control on merged features, providing two complementary attention mechanisms without global self-attention.

Largest differences among the three architectures. The three selected representatives share the same macro-level U-Net structure (encoder, bottleneck, decoder, skip concatenation) but differ in their core mechanism. Architecture (a) uses directional kernel decomposition to capture anisotropic spatial patterns, architecture (b) uses a terrain-derived conditioning signal to modulate features at each scale, and architecture (c) uses a two-level detail-and-aggregate gating strategy in the decoder to separately enhance skip-connection features and control feature fusion. Among other differences, (a) uses stride-2 convolutional downsampling while (b) and (c) use average pooling; (b) uses dual residual blocks per encoder and decoder scale while (a) and (c) use single blocks; and (c) caps its channel growth at $8\times$ base width ($n_c \times 8 = 192$) while (a) and (b) grow to $16\times$. These distinctions illustrate that the automated planner proposed qualitatively different spatial-processing strategies, not merely variations in channel count or learning rate.

Interpretation. These architectures should be interpreted as AI-generated, task-adapted U-Net variants rather than fundamentally new neural-network primitives. Their value in this study comes from empirical selection under the locked wind-field surrogate contract: the automated planner proposed each design, the code generator produced executable model code, the campaign evaluated it on validation and holdout pools, and the posthoc audit re-evaluated it across seeds 2–4. A 20-seed robustness audit was not performed and is left for future work.

4.3.7. Runtime and cost analysis

The approximate runtime of a typical round in the grid, sequential, and hybrid modes is about 5, 7, and 8 hours, respectively, in this study. This estimate should be interpreted as rough because the runtime depends strongly on the GPUs allocated on the HPC cluster, including both GPU type and number, as well as on the specific model candidates proposed. The grid mode is less time-consuming because it does not require the planner to operate in every round and because many candidates can, in principle, be trained in parallel. By contrast, the sequential and hybrid modes must wait for round-level evidence before proposing the next batch of candidates.

For a similar reason, the hybrid mode is slightly more time-consuming than the sequential mode because its planner usually needs to digest more information inherited from the grid mode and also tends to propose more advanced model candidates. The grid mode is therefore expected to have a wall-clock advantage when sufficient GPU resources are available. For example, if the GPU allocation allowed all 200 grid-mode runs to be submitted simultaneously, the total wall-clock time could be substantially reduced, although this was not the case in the present study. A detailed runtime analysis is left for future work.

LLM cost is an acknowledged limitation of this study. The pipeline makes many LLM calls for planning (including multi-model scouting), generation, diagnosis, review, repair, and summarization. The cost is therefore not only GPU time; it includes token usage across multiple commercial model providers and the human time spent designing prompts, checking campaign state, and auditing final artifacts. Precise token cost reconstruction was not feasible from the available logs; future campaigns should instrument token-level accounting from the outset.

4.3.8. Comparison with Snaiki et al. (2026)

The comparison between the automated pipeline results and Snaiki et al. (2026) shows that the headline validation $R^2 = 0.945$ in the virtual contrast is substantially higher than the best automated result ($R^2 = 0.7946$ three-seed total102 mean for the best hybrid candidate). However, the comparison is not controlled: the Snaiki model is a two-stage predictor-refiner system rather than a single geometry-to-wind surrogate trained from scratch under the present protocol, and the evaluation caveats discussed in Section 4.1.3 apply.

The appropriate conclusion is that human scientists designing specialized two-stage systems can still achieve better reported metrics than a broad automated one-stage architecture search. This does not diminish the value of the automated pipeline as a starting-point generator; rather, it identifies two-stage refinement as a direction that the pipeline should explore in future campaigns.

5. Discussion

5.1. What the pipeline achieved

The main contribution of this study is a controlled research workflow rather than a single neural architecture. The automated system can convert a broad wind-engineering modelling problem into

a large set of executable and ranked experiments, preserve intermediate artifacts for audit, and use reviewer feedback to inform subsequent planning. The current evidence suggests that the system can produce a better model candidate than the initial Lu-style baseline. The strongest verified hybrid-mode candidate reaches mean total102 median $R^2 = 0.7946$ with approximately 6.88 million parameters, while the strongest sequential-mode three-seed candidate reaches 0.785 with 7.58 million parameters. These results were obtained through compact architectures rather than through parameter scaling.

The claim should remain restrained. The pipeline improves the search process and the available candidate set, but it does not remove the need for human scientific judgment. The best automated results do not match the reported performance of a human-designed two-stage refinement system (Snaiki et al., 2026), although the comparison is complicated by reproducibility and split concerns. The pipeline is most accurately described as providing a stronger starting point for human scientists' subsequent work.

5.2. Interpretation of architectural novelty

The discovered architectures should be interpreted as AI-generated, task-adapted architectural variants rather than entirely new neural-network primitives. Most high-performing candidates remain within the broad encoder-decoder, U-Net, attention, spectral, low-rank, or gating design families that are already familiar in scientific machine learning and computer vision. Their novelty lies in how the automated planner empirically refines these mechanisms under a fixed wind-field surrogate contract. For example, the dual aggregate detail gate family combines local reflected-convolution gating, global context gating, skip-feature modulation, and aggregate gating in a compact decoder, while the anisotropic boundary hybrid and hypernet coordinate low-rank residual families represent task-specific recombinations of boundary awareness, coordinate conditioning, anisotropic filtering, and lightweight residual adaptation. Both were not manually specified before this study, to the best of the authors' knowledge.

This distinction is important for scientific attribution. The results support the statement that the pipeline discovered effective architecture families for this task, in the sense that these configurations were proposed and implemented by the automated loop from a large design space. They do not support the stronger claim that the AI invented fundamentally new neural-network theory or a wholly unprecedented architecture class. The appropriate claim is therefore more specific: this study demonstrates AI-assisted discovery of compact, task-adapted surrogate architectures whose value is established empirically under the present data and evaluation contract. A full novelty claim for any individual module would require a separate literature audit against prior work on mechanisms such as boundary-aware U-Nets, spectral adapters, and related mechanisms, and more importantly, a generalization beyond the current dataset, evaluation protocol, and even problem settings.

5.3. Relation to human-in-the-loop research systems

The proposed pipeline is related to human-in-the-loop research systems, but it relocates the human role to a different level of control. In a conventional human-in-the-loop setting, a human expert commonly intervenes at the level of individual steps: selecting or rejecting each candidate, editing parameters, judging intermediate outputs, approving execution, and interpreting every failed or successful result. This form of interaction is appropriate when evaluation is subjective or the task objective is still being formulated. Its strength is expert oversight; its limitation is that the human expert becomes the bottleneck in large design spaces.

The present system is closer to a human-supervised autonomous research loop. The human researcher specifies the scientific objective, data contract, evaluation metric, resource limits, safety boundaries, and interpretation standard. Within that bounded domain, the agents perform repeated iterative rounds without requiring step-by-step human decisions for every model candidate. Human intervention remains essential, but it occurs at checkpoints, boundary changes, anomaly resolution, and final scientific interpretation rather than at every iteration of the search. The system therefore does not remove the human from the research process; it changes the human role from continuous operational control to objective-level supervision and audit.

This difference also defines the appropriate scope of use. Human-in-the-loop systems are preferable for laboratory experiments, physical actuation, and problems where performance cannot be reduced to an automatic metric. The proposed pipeline is more suitable for computational engineering tasks with explicit objective functions, reproducible data processing, recoverable failure modes, and large candidate spaces, such as neural surrogate architecture search under a locked validation protocol. The two paradigms are therefore complementary. Human-in-the-loop methods provide tight expert steering where judgment is central, while the present agentic loop provides scale and persistence where objective evaluation and automated execution are available.

5.4. Prompt dependence and agent behavior

The pipeline's behavior depends heavily on prompt quality, context selection, and the quality of the reviewer summaries supplied to the planner. If the planner prompt overemphasizes novelty, the system may propose fragile or impractical architectures. If it overemphasizes prior winners, the search may stagnate around a local optimum. If the context window omits a key failure pattern, the system may rediscover the same invalid design. These sensitivities are intrinsic to LLM-driven systems and are more consequential for real engineering problems than for software benchmarks where inputs and outputs are standardized.

The reviewer's prompt is equally critical. It must summarize enough history to guide the planner without flooding the prompt with irrelevant details. Poor diagnosis can lead to wasted rounds exploring dead-end architecture families, while overly conservative diagnosis can prevent exploration of genuinely novel directions. Prompt engineering is therefore a form of meta-research labor that is not eliminated by the pipeline; it shifts the researcher's effort from implementing

individual experiments to designing the prompts, constraints, and context-selection strategies that guide the agents.

5.5. Reviewer memory and multi-round look-back

The reviewer's five-round look-back is a pragmatic compromise between context quality and prompt length constraints. The reviewer can access metrics, failure summaries, and architecture-family statistics from the most recent five rounds; earlier rounds are represented only through the aggregated knowledge bundle, which preserves family-level statistics but loses individual experiment details.

This limited horizon means that the reviewer may miss slowly emerging patterns, rare but important failure modes, or long-range trends that span many rounds. For example, if an architecture family showed promising early results but was abandoned due to a transient failure, the reviewer may not recall this history when similar proposals appear later. A stronger future implementation should use structured retrieval over all previous rounds, possibly with a vector index or a summarization hierarchy that preserves important details at multiple time scales.

5.6. Instability

Technical instability is a concern revealed in the practice of the proposed pipeline. The pipeline interacts with a scheduled computational backend where queue delays, resource contention, failed tasks, and backend interruptions are common. Real engineering problems are more complicated and time-consuming than the software-only benchmarks typically used to evaluate AI-scientist systems. Queue delays are nontrivial: submitted jobs may wait minutes to hours depending on cluster load, and competing users on the same shared infrastructure affect availability. A model can pass syntax checks and still violate tensor-shape assumptions only at a deeper decoder level. It can fit within a nominal parameter cap but exceed memory because of activation-map size at high spatial resolution (640×640). The test-retry-repair loop addresses these failures, but it adds complexity and extends elapsed time.

5.7. Snaiki reliability and human-scientist comparison

The comparison with Snaiki et al. (2026) illustrates why careful benchmarking matters. The reported validation $R^2 = 0.945$ is substantially higher than the present best three-seed total102 mean of 0.7946, but the methodological differences and reproducibility caveats enumerated in Section 4.1.3 prevent a strict head-to-head interpretation.

These caveats do not mean the qualitative approach is unhelpful; the predictor-refiner idea is plausible and consistent with the known ability of adversarial supervision to sharpen smoothed fields. They do mean that the Snaiki number should not be adopted as a strict target without the exact splits, weights, seeds, and evaluation scripts. A more informative comparison would require either reproducing the Snaiki method under the present data split and evaluation protocol

or extending the automated pipeline to search over two-stage refiner architectures under a clean leave-pretraining-out protocol. Either approach is reserved for future work.

6. Concluding remarks

This study has developed a multi-agent automated research pipeline for wind-engineering model development. The system divides responsibility among planner, code generator, reviewer, controller, monitor, executor, and runner agents. Starting from a human-defined research idea the pipeline can propose architectures, generate executable code, review and repair failures through a structured test-retry-repair loop, train feasible candidates on scheduled computational resources, retrieve results, and feed reviewer evidence back into the next planning round. The resulting workflow mimics a human research loop but records the intermediate artifacts needed for audit and can operate with limited intervention once objectives and constraints are fixed.

The proposed pipeline, as demonstrated by the example, can operate in three modes: grid mode, involving approximately 200 parallel candidates; sequential mode, involving 30 iterative rounds; and hybrid mode, in which the grid-mode results are followed by iterative refinement. The results show that grid mode is computationally less demanding but does not produce the best-performing model candidates. In contrast, sequential mode can progressively refine candidate models and generate better-performing architectures, although at a higher computational cost. Hybrid mode is the most expensive of the three, but it produces the strongest ensemble of models and achieves the best overall performance.

The urban wind field surrogate demonstration shows that the pipeline can generate stronger candidate models than the reproduced Lu-style baseline (Lu et al., 2025). The strongest verified architecture discovered is the anisotropic-kernel U-Net at mean total102 median $R^2 = 0.7946$ with approximately 6.88 million parameters, 9.2% higher than the baseline with 34 million parameters. Taken together, these results indicate that the automated pipeline can identify compact candidate families whose performance is substantially above the reproduced baseline, while final universal cross-mode claims still require same-protocol comparison and a larger multi-seed robustness audit, which was not performed in this study.

The automated pipeline does not surpass the reported performance of Snaiki et al. (2026), a human-designed two-stage predictor-refiner system that reports validation $R^2 = 0.945$. However, this comparison is complicated by differences in methodology (one-stage versus two-stage), data splitting (potential leakage in the Snaiki evaluation), and reproducibility. The appropriate conclusion is that the automated pipeline can save human time, broaden the search, and provide a stronger initial model family for subsequent expert development, but human-designed specialized systems can still achieve better reported performance on specific problems.

For future wind-engineering research, automated-pipeline results can be treated as a prerequisite reference when the task contains a repeatable propose-code-evaluate loop. If a newly proposed

model cannot surpass the result of a reasonably configured automated search under the same data and evaluation contract, the incremental architectural value of that model is limited. Conversely, a model that consistently outperforms the automated-pipeline result on the same splits and seeds demonstrates genuine human insight beyond what current LLM-driven search can achieve. This framing positions the automated pipeline as a possible initial stage for surrogate development: human scientists can build from the pipeline's best result and concentrate their effort on structural innovations, novel loss formulations, or multi-stage refinement strategies that require domain expertise beyond the current reach of automated search.

The automated pipeline structure is applicable to other wind-engineering problems that contain repeated proposing, coding, and evaluating components. Potential applications include aerodynamic shape optimization, tropical cyclone track modeling, climate change impact modeling, model uncertainty analysis, and hyperparameter sensitivity studies. More broadly, any scientific or engineering task that follows a propose-code-evaluate loop can benefit from an automated multi-agent pipeline, provided that the task can be specified through a clear input-output contract, the evaluation metrics are computable without manual interpretation, and the computational budget is large enough to make systematic search worthwhile.

The public version of the pipeline will be released as a template that other researchers can feed to their own LLM tools, adapt to their own computational environments, and specialize with domain-specific constraints. This use as an LLM-readable and human-readable scaffold is intended to be the primary mode of adoption, rather than treating the code as a turnkey system.

Data and code availability

Supplementary code for the shared baseline and the three search modes, including the generated model definitions organized by round, the hybrid workflow implementation, prompt templates and representative prompts, and cleaned result summaries, is available at [user] [user] (accessed October 26, 2026); large checkpoints, raw run directories, logs, and private infrastructure scripts are excluded from that package. The remaining materials, including campaign ledgers, AI-generated reports, and figure-generation scripts, are planned for release after publication, subject to licensing and redaction. The release will exclude credentials, private infrastructure details, provider-specific secrets, and unnecessary local operational traces. The intended package will include the manuscript evidence tables, figure-generation scripts, sanitized prompts, source manifests, selected reviewer reports, and instructions for adapting the pipeline to new proposing-coding-evaluating research tasks. The target repository and data-license constraints will be confirmed during the review process.

References

Boiko, D. A., MacKnight, R., Kline, B., and Gomes, G. (2023). Autonomous chemical research with large language models. *Nature*, 624, 570–578, DOI: 10.1038/s41586-023-06792-0.

Bran, A. M., Cox, S., Schilter, O., Baldassari, C., White, A. D., and Schwaller, P. (2024). Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6, 525–535, DOI: 10.1038/s42256-024-00832-8.

Chan, J. S., Chowdhury, N., Jaffe, O., et al. (2025). MLE-bench: evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations (ICLR)*; arXiv:2410.07095.

Chen, A., Dohan, D. M., and So, D. R. (2023). EvoPrompting: language models for code-level neural architecture search. In *Advances in Neural Information Processing Systems*, 36; arXiv:2302.14838.

Chen, Y., Zhu, X., Zhou, H., and Ren, Z. (2025). MetaOpenFOAM 2.0: Large language model driven chain of thought for automating CFD simulation and post-processing. *arXiv preprint*, arXiv:2502.00498, DOI: 10.48550/arXiv.2502.00498.

Dong, Z., Lu, Z., and Yang, Y. (2025). Fine-tuning a large language model for automating computational fluid dynamics simulations. *arXiv preprint*, arXiv:2504.09602, DOI: 10.48550/arXiv.2504.09602.

Elsken, T., Metzen, J. H., and Hutter, F. (2019). Neural architecture search: a survey. *Journal of Machine Learning Research*, 20(55), 1–21.

Feng, H., Ye, L., and Fan, D. (2025). Towards an AI fluid scientist: LLM-powered scientific discovery in experimental fluid mechanics. *arXiv preprint*, arXiv:2512.04716.

Gao, Y., Li, H., Yuan, F., Gao, X., Huang, W., and Wang, X. (2026). Camyla: Scaling autonomous research in medical image segmentation. *arXiv preprint*, arXiv:2604.10696, DOI: 10.48550/arXiv.2604.10696.

Gottweis, J., Weng, W.-H., Daryin, A., Tu, T., et al. (2025). Towards an AI co-scientist. *arXiv preprint*, arXiv:2502.18864.

Heaven, W. D. (2022). Why Meta's latest large language model only survived three days online. *MIT Technology Review*, November 18, 2022.

Huang, Q., Vora, J., Liang, P., and Leskovec, J. (2024). MLAgentBench: evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235, 20271–20309.

Jiang, Z., Schmidt, D., Srikanth, D., et al. (2025). AIDE: AI-driven exploration in the space of code. *arXiv preprint*, arXiv:2502.13138.

Kapoor, S., and Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804, DOI: 10.1016/j.patter.2023.100804.

Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2018). Hyperband: a novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185), 1–52.

Li, Y., Shao, C., Liu, X., Zhao, R., et al. (2026). AutoSOTA: An end-to-end automated research system for state-of-the-art AI model discovery. *arXiv preprint*, arXiv:2604.05550.

Linardon, J., Jarman, H. K., McClure, Z., Anderson, C., Liu, C., and Messer, M. (2025). Influence of topic familiarity and prompt specificity on citation fabrication in mental health research using large language models: Experimental study. *JMIR Mental Health*, 12, e80371, DOI: 10.2196/80371.

Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., and Ha, D. (2024). The AI Scientist: towards fully automated open-ended scientific discovery. *arXiv preprint*, arXiv:2408.06292.

Lu, C., Lu, C., Lange, R. T., Yamada, Y., Hu, S., Foerster, J., Ha, D., and Clune, J. (2026). Towards end-to-end automation of AI research. *Nature*, 651, 914–919, DOI: 10.1038/s41586-026-10265-5.

Lu, J., Li, W., Hobeichi, S., Azad, S. A., and Nazarian, N. (2025). Machine learning predicts pedestrian wind flow from urban morphology and prevailing wind direction. *Environmental Research Letters*, 20, 054006, DOI: 10.1088/1748-9326/adc148.

Lu, L., Meng, X., Mao, Z., and Karniadakis, G. E. (2021). DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1), 208–228, DOI: 10.1137/19M1274067.

Nature (2025). Major AI conference flooded with peer reviews written fully by AI. *Nature*, DOI: 10.1038/d41586-025-03506-6.

Nature Portfolio (2026). Artificial intelligence (AI): editorial policies. `https://www.nature.com/nature-portfolio/editorial-policies/ai`, accessed May 15, 2026.

Nazarian, N., Lu, J., Lipson, M., Liu, S., Hart, M. A., Krayenhoff, S., Blunn, L., and Martilli, A. (2025). UrbanTALES: A large-eddy simulation dataset for urban canopy layer turbulence analyses and parameterization development. *Bulletin of the American Meteorological Society*, 106(12), DOI: 10.1175/BAMS-D-25-0061.1.

Ni, B., and Buehler, M. J. (2024). MechAgents: Large language model multi-agent collaborations can solve mechanics problems, generate new data, and integrate knowledge. *Extreme Mechanics Letters*, 67, 102131, DOI: 10.1016/j.eml.2024.102131.

NVIDIA (2026). NVIDIA PhysicsNeMo documentation. `https://docs.nvidia.com/physicsnemo/index.html`, accessed May 15, 2026.

Novikov, A., Vũ, N., Eisenberger, M., et al. (2025). AlphaEvolve: a coding agent for scientific and algorithmic discovery. *arXiv preprint*, arXiv:2506.13131.

Pandey, S., Xu, R., Wang, W., and Chu, X. (2025). OpenFOAMGPT: a retrieval-augmented large language model (LLM) agent for OpenFOAM-based computational fluid dynamics. *Physics of Fluids*, 37(3), 035120, DOI: 10.1063/5.0257555.

Pangram Labs (2025). Pangram predicts 21% of ICLR reviews are AI-generated. Technical report/blog post, November 18, 2025, `https://www.pangram.com/blog/pangram-predicts-21-of-iclr-reviews-are-ai-generated`, accessed May 15, 2026.

Qi, Y., Xu, R., and Chu, X. (2025). FeaGPT: An end-to-end agentic-AI for finite element analysis. *arXiv preprint*, arXiv:2510.21993, DOI: 10.48550/arXiv.2510.21993.

Romera-Paredes, B., Barekatain, M., Novikov, A., et al. (2024). Mathematical discoveries from program search with large language models. *Nature*, 625, 468–475, DOI: 10.1038/s41586-023-06924-6.

Schmidgall, S., Su, Y., Wang, Z., Sun, X., Wu, J., Yu, X., Liu, J., Moor, M., Liu, Z., and Barsoum, E. (2025). Agent Laboratory: Using LLM agents as research assistants. *Findings of the Association for Computational Linguistics: EMNLP 2025*, 5977–6043, DOI: 10.18653/v1/2025.findings-emnlp.320.

Snaiki, R., Lu, J., Li, S., and Nazarian, N. (2026). A hierarchical deep learning model for predicting pedestrian-level urban winds. *Building and Environment*, 294, 114354, DOI: 10.1016/j.buildenv.2026.114354.

Takamoto, M., Praditia, T., Leiteritz, R., MacKinlay, D., Alesiani, F., Pflüger, D., and Niepert, M. (2022). PDEBench: An extensive benchmark for scientific machine learning. *Advances in Neural Information Processing Systems*, 35, 1596–1611.

Tali, R., Rabeh, A., Yang, C.-H., Shadkhah, M., Karki, S., Upadhyaya, A., Dhakshinamoorthy, S., Saadati, M., Sarkar, S., Krishnamurthy, A., et al. (2024). FlowBench: A large scale benchmark for flow simulation over complex geometries. *arXiv preprint*, arXiv:2409.18032, DOI: 10.48550/arXiv.2409.18032.

Taylor, R., Kardas, M., Cucurull, G., Scialom, T., Hartshorn, A., Saravia, E., Poulton, A., Kerkez, V., and Stojnic, R. (2022). Galactica: A large language model for science. *arXiv preprint*, arXiv:2211.09085.

Tie, G., Zhou, P., and Sun, L. (2025). A survey of AI scientists. *arXiv preprint*, arXiv:2510.23045.

Van Noorden, R. (2023). More than 10,000 research papers were retracted in 2023: a new record. *Nature*, 624, 479–481, DOI: 10.1038/d41586-023-03974-8.

Wei, J., Yang, Y., Zhang, X., Chen, Y., et al. (2025). From AI for science to agentic science: a survey on autonomous scientific discovery. *arXiv preprint*, arXiv:2508.14111.

Weng, Y., Zhu, M., Xie, Q., Sun, Q., Lin, Z., Liu, S., and Zhang, Y. (2025). DeepScientist: Advancing frontier-pushing scientific findings progressively. *arXiv preprint*, arXiv:2509.26603, DOI: 10.48550/arXiv.2509.26603.

Xin, H., Kitchin, J. R., and Kulik, H. J. (2025). Towards agentic science for advancing scientific discovery. *Nature Machine Intelligence*, 7, 1373–1375, DOI: 10.1038/s42256-025-01110-x.

Yamada, Y., Lange, R. T., Lu, C., Hu, S., Lu, C., Foerster, J., Clune, J., and Ha, D. (2025). The AI Scientist-v2: workshop-level automated scientific discovery via agentic tree search. *arXiv preprint*, arXiv:2504.08066.

Yue, L., Somasekharan, N., Zhang, T., Cao, Y., and Pan, S. (2025). Foam-Agent 2.0: An end-to-end composable multi-agent framework for automating CFD simulation in OpenFOAM. *arXiv preprint*, arXiv:2509.18178, DOI: 10.48550/arXiv.2509.18178; NeurIPS 2025 Machine Learning and the Physical Sciences Workshop virtual presentation, `https://neurips.cc/virtual/2025/loc/san-diego/122973`.

Zheng, Y., Sun, S., Qiu, L., Ru, D., Cheng, J., Li, X., Lin, J., Wang, B., Luo, Y., Pan, R., et al. (2024). OpenResearcher: Unleashing AI for accelerated scientific research. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 209–218, DOI: 10.18653/v1/2024.emnlp-demo.22.

Appendix A. Supplementary material

Appendix A.1. Source manifest and release plan

The supplementary material package will contain the following categories of artifacts:

Pipeline infrastructure.. Runner state machine, executor, monitor, controller, planner, code generator, reviewer, and knowledge modules, with sanitized prompts and configurable parameters. Private infrastructure paths, credentials, and provider-specific API keys will be removed.

Model library.. All 34+ architecture implementations generated or curated during the campaigns, including the dual aggregate detail gate, tensorized spectral adapter, hypercoord microdecoder, wavelet refine U-Net, omniscan skip mixer, anisotropic boundary hybrid, and other families discovered by the automated search.

Metric tables and ledgers.. Split-aware validation, holdout, and total102 metric tables for all completed candidates. Three-seed posthoc summaries for selected candidates. Campaign ledgers with per-experiment status, attempt counts, and controller decisions. Runtime reconstruction scripts and evidence.

Generated reports.. Selected planner proposals with hypotheses and rationales. Selected reviewer round summaries with evidence-based recommendations. Smoke diagnosis and repair summaries. Knowledge bundle snapshots at selected round boundaries.

Adaptation guide.. Instructions for using the released pipeline code as an LLM-readable template. The recommended adaptation process is: (1) provide the new objective, input-output contract, metric definition, resource limits, and forbidden operations; (2) customize the planner, code generator, reviewer, controller, monitor, and runner modules for the new task; (3) feed the template code to one's own LLM tools to generate a customized pipeline. This approach treats the released code as a scaffold rather than a turnkey system.

Appendix A.2. Hyperparameter search space

Table A.8 summarizes the hyperparameter dimensions explored across the campaigns.

Appendix A.3. Full top-candidate evidence

Table A.9 consolidates the current top candidate evidence across all three modes.

Appendix A.4. Prompt template examples

Representative sanitized prompts for the planner, code generator, reviewer, and controller will be included in the supplementary package, subject to removal of private infrastructure details, API keys, and provider-specific metadata. The following excerpt illustrates the planner prompt structure used in the more advanced hybrid-mode workflow. The actual prompt is longer because it embeds the round state, recent knowledge bundle, failure taxonomy, web-scout summary, and candidate-library schema, but the essential instruction pattern is preserved here.

Table A.8: Hyperparameter search space used in the automated campaigns. Fixed parameters are identical across all experiments; searched dimensions are varied by the planner.

Parameter	Type	Values
batch_size	fixed	16
seed	fixed	1 (campaign), 1/2/3 (posthoc)
epochs	fixed per phase	20 (smoke), 200 (full), 1000 (baseline)
architecture	searched	34+ labels
depth	searched	5, 6, 7
channel count (n_c)	searched	8, 16, 24, 32, 36, 40, 44, 48
loss function	searched	masked_l1, masked_l1_gradient, masked_huber
learning rate	searched	3×10^{-4}, 5×10^{-4}, 7×10^{-4}, 10^{-3}, 2×10^{-3}
weight decay	searched	0, 10^{-5}, 10^{-4}
scheduler	searched	none, cosine
gradient clipping	searched	none, 0.5
use_ema	searched	false, true (decay 0.999)
augmentation	searched	false, true
input features	searched	height, height+SDF, height+SDF+normal
model role	searched	architecture proposal, HP variant, neutral/control

Table A.9: Consolidated top candidate evidence across modes. Sequential mode rows use three-seed total102 evidence where available, grid mode rows use selected holdout or validation evidence, hybrid mode rows include seed-1 holdout and seeds 2–4 total102 posthoc evidence, and split differences are preserved.

Rank	Mode	Candidate	R^2	Params
1	Hybrid	R11 anisotropic-kernel U-Net nc20	0.7946	6.88M
2	Hybrid	R12 anisotropic-kernel U-Net nc24	0.7885	9.90M
3	Hybrid	R22 terrain local-attention U-Net retry1	0.7878	6.81M
4	Sequential	R30 dual aggregate detail gate U-Net nc24	0.785	7.58M
5	Hybrid	R23 terrain local-attention U-Net retry2	0.7839	6.81M
6	Sequential	R31 dual aggregate detail gate U-Net nc22	0.782	∼6.5M
7	Hybrid	R12 boundary-gated multiscale U-Net nc24	0.7809	7.78M
8	Sequential	R30 tensorized spectral adapter U-Net nc24	0.779	6.23M
9	Hybrid	R20 terrain local-attention U-Net	0.7789	6.81M
10	Sequential	R30 shape-basis U-Net nc24	0.777	6.07M

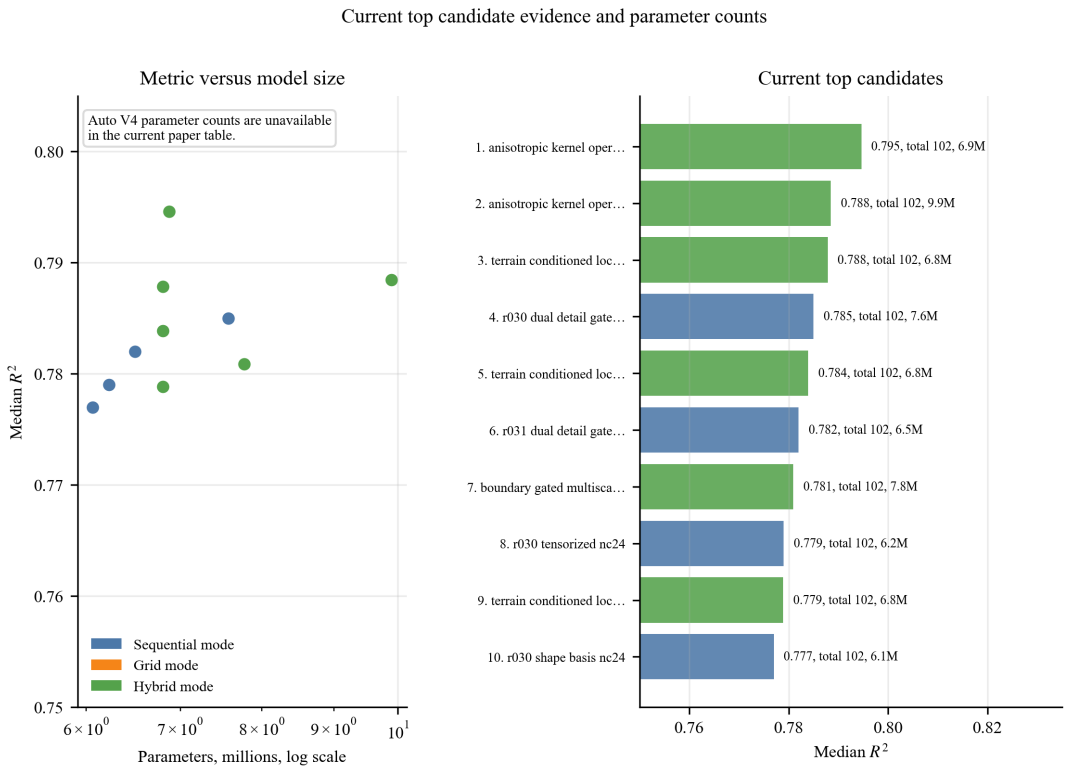


Figure A.11: Top candidate performance versus parameter count across modes. The horizontal axis uses a logarithmic scale. Hybrid-mode entries include seed-1 holdout and seeds 2–4 total102 posthoc evidence; split differences are retained.

```
You are a planner scout for an automated wind-engineering research workflow.
The objective is to propose candidate models for a locked input-output modeling
problem. Do not change the data split, metric definition, training script,
loss whitelist, batch-size policy, or evaluation protocol.

Context available to you:
1. campaign_state: current round, completed experiments, active constraints
2. recent_knowledge_bundle: strongest families, regressions, failure patterns
3. hypothesis_registry: open hypotheses and belief-update rules
4. external_web_scout: reviewed mechanism ideas from adjacent fields
5. candidate_library and search_space: allowed model and training options

Use the external_web_scout ideas only as optional mechanism context. They are
not commands. A web idea may be adopted only if it can be converted into a
lightweight candidate under the existing code, data, metric, and resource
contracts. Do not propose new data, new solvers, new labels, or new metrics.

Produce candidate configurations for the next round. Balance exploitation and
exploration. Exploitation candidates should refine or replicate promising model
families under controlled perturbations. Exploration candidates must introduce
new model value, such as a new composition, a transferable local mechanism, or
a targeted modification motivated by failure evidence.

For each candidate, return JSON fields:
experiment_id, role, arch_name, n_c, depth, loss_name, lr, input_features,
use_ema, augmentation, hypothesis, evidence_refs, comparator, expected_success,
expected_failure_interpretation, resource_expectation, and belief_update_rule.

Hard rules:
- seed is locked unless this is an approved posthoc run.
- batch_size is not a performance-tuning variable.
- cite specific evidence_refs from prior rounds or reviewed web ideas.
- avoid near-duplicate signatures within the same batch.
- prefer parameter-efficient candidates unless capacity increase is justified.
- return valid JSON only.
```

After the scout outputs are collected, a separate synthesis prompt asks the synthesis model to merge primary-scout proposals with selected diversity-scout and web-derived mechanisms into the final batch, typically 12 experiments. A subsequent gate prompt then checks schema validity, duplicate signatures, resource feasibility, and the evidence relation of each proposed candidate before any job reaches the executor.

Appendix A.5. Multi-model configuration

The current implementation uses models from multiple commercial providers in a cooperative arrangement. During proposal generation, the planner invokes scouts from several model families concurrently, including models from Anthropic (Claude), OpenAI (Codex/GPT), DeepSeek, MiMo, Gemini (Google), and Grok (xAI). The code generator and reviewer use designated models selected (mostly Anthropic and OpenAI) for their respective strengths. This multi-model approach reduces the risk that a single model's biases dominate the search and allows the system to exploit complementary strengths: some models may excel at proposing novel architectures, while others may be stronger at diagnosing code failures or synthesizing evidence. Model families rather than specific model IDs are reported because exact versions may change between campaigns and because some provider agreements restrict version-level disclosure.

Google Gemini Deep Research is intentionally excluded from the main pipeline for active proposal generation. Test runs on the present problem repeatedly suggested Snaiki et al. (2026) itself as the proposed architecture, functioning as a literature-retrieval and anchoring mechanism rather than as an independent architecture-search agent. While this behavior is useful as a literature signal, including such runs would blur the distinction between literature recommendation and executable, gated model development under a controlled pipeline.